Spring 2025 Homework Modeling Exercises Weeks 1 & 3

Group 2: Bailey Meche, Eliott Raia, University of Chicago

04-07-2026

Download data

3. Set RStudio's working directory to the folder containing the datasets for this homework and open the PJM data set ("data_PJM.csv"), as below. This dataset is constructed from hourly pricing data from PJM's Data Miner 2 and includes both day-ahead ("DA") and real-time("RT") locational marginal pricing ("LMP") data.

```
data_PJM <- fread(paste0(wd,"/", "PJM Data.csv"))
```

4. Load the following data set, which combines pricing and weather data for Chicago, Milwaukee, and Corpus Christi.

```
data_Chicago <- fread(paste0(wd,"/",'Data Chicago.csv'))
data_Milwaukee <- fread(paste0(wd,"/",'Data Milwaukee.csv'))
data_CorpusChristi<- fread(paste0(wd,"/",'Data Corpus Christi.csv'))
data_Austin <- fread(paste0(wd,"/",'Data Austin.csv'))
```

III. Data Familiarization

5. Let's explore the PJM data! The following code identifies the fields of the dataset

```
names(data_PJM)
```

```
##  [1] "datetime_beginning_utc" "datetime_beginning_ept" "pnode_id"
##  [4] "pnode_name"             "voltage"                "equipment"
##  [7] "type"                   "zone"                   "system_energy_price_rt"
## [10] "total_lmp_rt"           "congestion_price_rt"    "marginal_loss_price_rt"
## [13] "system_energy_price_da" "total_lmp_da"           "congestion_price_da"
## [16] "marginal_loss_price_da" "spread"                 "date"
## [19] "hour"                   "am_pm"                  "day"
## [22] "day_of_year"            "month_num"              "month"
## [25] "year"                   "day_of_week"            "weekend"
## [28] "isHoliday"              "holiday"
```

6. To see the first few observations of the data, run the code below. What is the unit ofobservation?

Each row represents one hour of pricing data for one specific pricing node. The unit of observation is: one pricing node × one hour.

```
head(data_PJM)
```

```
##    datetime_beginning_utc datetime_beginning_ept pnode_id pnode_name voltage
##                    <char>                 <char>    <i64>     <char>  <lgcl>
## 1:    2/1/2021 4:00:00 AM  1/31/2021 11:00:00 PM        1        PJM      NA
## 2:    2/1/2021 4:00:00 AM  1/31/2021 11:00:00 PM    51205  BRANDONSH      NA
## 3:    2/1/2021 4:00:00 AM  1/31/2021 11:00:00 PM    51206  BRUNSWICK      NA
```

```
## 4:    2/1/2021 4:00:00 AM  1/31/2021 11:00:00 PM    51211  COOKSTOWN      NA
## 5:    2/1/2021 4:00:00 AM  1/31/2021 11:00:00 PM    51214      DOVER      NA
## 6:    2/1/2021 4:00:00 AM  1/31/2021 11:00:00 PM    51215  DPL NORTH      NA
##    equipment      type   zone system_energy_price_rt total_lmp_rt
##       <lgcl>    <char> <char>                  <num>        <num>
## 1:        NA      ZONE                         25.06     25.03782
## 2:        NA AGGREGATE                         25.06     25.40750
## 3:        NA AGGREGATE                         25.06     21.82667
## 4:        NA AGGREGATE                         25.06     18.05833
## 5:        NA AGGREGATE                         25.06     20.76083
## 6:        NA AGGREGATE                         25.06     20.29252
##    congestion_price_rt marginal_loss_price_rt system_energy_price_da
##                  <num>                  <num>                  <num>
## 1:           -0.037294               0.015118                  21.31
## 2:           -1.106667               1.454167                  21.31
## 3:           -4.904167               1.670833                  21.31
## 4:           -8.830000               1.828333                  21.31
## 5:           -6.325000               2.025833                  21.31
## 6:           -6.316337               1.548852                  21.31
##    total_lmp_da congestion_price_da marginal_loss_price_da    spread       date
##           <num>               <num>                  <num>     <num>     <IDat>
## 1:     21.35672           -0.205410               0.252130 -3.681104 2021-01-31
## 2:     21.16000           -1.640000               1.490000 -4.247500 2021-01-31
## 3:     21.58000           -1.810000               2.080000 -0.246667 2021-01-31
## 4:     19.94000           -3.590000               2.220000  1.881667 2021-01-31
## 5:     20.91000           -2.750000               2.350000  0.149167 2021-01-31
## 6:     20.49766           -2.755891               1.943547  0.205141 2021-01-31
##     hour  am_pm   day day_of_year month_num  month  year day_of_week weekend
##    <int> <char> <int>       <int>     <int> <char> <int>      <char>  <lgcl>
## 1:    23     PM    31          31         1    Jan  2021      Sunday    TRUE
## 2:    23     PM    31          31         1    Jan  2021      Sunday    TRUE
## 3:    23     PM    31          31         1    Jan  2021      Sunday    TRUE
## 4:    23     PM    31          31         1    Jan  2021      Sunday    TRUE
## 5:    23     PM    31          31         1    Jan  2021      Sunday    TRUE
## 6:    23     PM    31          31         1    Jan  2021      Sunday    TRUE
##    isHoliday holiday
##       <lgcl>  <char>
## 1:     FALSE
## 2:     FALSE
## 3:     FALSE
## 4:     FALSE
## 5:     FALSE
## 6:     FALSE
```

7. The code below identifies the final date of the PJM data as December, 12, 2023. How wouldyou identify the initial date?

Initial date 2021-01-01 Final date 2023-12-31 Coverage Exactly 3 full calendar years

```r
min(data_PJM$date)
```

```
## [1] "2021-01-01"
```

```r
max(data_PJM$date)
```

```
## [1] "2023-12-31"
```

8. The code below identifies the set of pricing node types in the PJM data and calculates how many observations correspond to each type. How would you do the same for pricing `nodesnames`?

There are 339 unique pricing node names. Most nodes have exactly 26,280 observations ($24 \times 365 \times 3 = 26{,}280$ hourly records), matching the full 2021–2023 window. A handful of nodes – SOUTH (22,657), FLINTRUN (8,016), and GREENECC (2,113) – have fewer observations, indicating they were added to the PJM system partway through the period.

```r
unique(data_PJM$type)
```

```
## [1] "ZONE"                 "AGGREGATE"            "HUB"
## [4] "EHV"                  "INTERFACE"            "RESIDUAL_METERED_EDC"
```

```r
data_PJM[,.N, by = type]
```

```
##                    type       N
##                  <char>   <int>
## 1:                 ZONE  578160
## 2:            AGGREGATE 4135890
## 3:                  HUB  315360
## 4:                  EHV 2795809
## 5:            INTERFACE  187583
## 6: RESIDUAL_METERED_EDC  709560
```

9. The code below identifies the number of unique holiday values. How would you identify thenumber of observations falling on holidays? How would you identify the number of uniquedifferent pricing nodes?

Unique holiday values 12 (e.g. Christmas, Thanksgiving, Labor Day . . .) Observations on holidays 262,872 Unique pricing node names 339

```r
length(unique(data_PJM$holiday))          # unique holiday types
```

```
## [1] 12
```

```r
nrow(data_PJM[isHoliday == TRUE])         # observations on holidays
```

```
## [1] 262872
```

```r
length(unique(data_PJM$pnode_name))       # unique pricing nodes
```

```
## [1] 339
```

```r
length(unique(data_PJM$holiday))
```

```
## [1] 12
```

10. The following code restricts the PJM data to only the "date" and "total_lmp_rt" fields. Whatfields would be interesting to study in conjunction?

Beyond date and RT LMP, some informative combinations include: • total_lmp_rt and total_lmp_da: the RT–DA spread reveals how real-time prices deviate from day-ahead forecasts, a key measure of market risk. • hour and total_lmp_rt: captures intraday price patterns (morning ramp, evening peak). • month or season and total_lmp_rt: reveals seasonal demand-driven price variation. • day_of_week and total_lmp_rt: identifies weekday vs. weekend price differences caused by industrial load patterns. • isHoliday and total_lmp_rt: isolates holiday demand suppression effects on prices. • pnode_name and congestion_price_rt: highlights which nodes experience the most transmission congestion.

```
data_PJM[, .(date, total_lmp_rt)]
```

```
##                date total_lmp_rt
##              <IDat>        <num>
##       1: 2021-01-31     25.03782
##       2: 2021-01-31     25.40750
##       3: 2021-01-31     21.82667
##       4: 2021-01-31     18.05833
##       5: 2021-01-31     20.76083
##      ---
## 8722358: 2023-12-31     20.45000
## 8722359: 2023-12-31     20.33167
## 8722360: 2023-12-31     20.21500
## 8722361: 2023-12-31     19.98767
## 8722362: 2023-12-31     20.45583
```

11. The code below creates a new dataset, restricted to PJM observations falling on Christmas.Create a dataset, named "holidays_PJM," consisting of all observations falling on a holiday.

holidays_PJM contains 262,872 rows, encompassing all hourly observations across all 12 recognized holidays in the dataset for each of the 339 pricing nodes.

```
xmas_PJM <- data_PJM[holiday == 'Christmas']
```

12. The code below calculates the average RT LMP on holidays in November and December, during AM hours, for the "'CHICAGO GEN HUB" pricing node.6 What observations arebeing selected? What is being averaged over? What stands out about the results? Write codethat averages DA LMP within each month, for the "Chicago Gen Hub." Formulate ahypothesis based on the result you observe. How could you check your hypothesis?

What observations are selected: Christmas Day hourly observations at the Chicago Gen Hub node, restricted to AM hours (hours 0–11), across all three years. This is an extremely small sample — at most 12 hours × 3 years = 36 observations.

What stands out: The year-over-year contrast is extreme and driven almost entirely by December 2022. In 2021, Christmas AM RT LMPs range roughly $15–28/MWh with a modest morning ramp. In 2022, the same hours range from $73–450/MWh — hour 0 (midnight) averages $450/MWh, a figure that reflects the December 2022 bomb cyclone, not a typical Christmas market. In 2023, prices are near or below zero for hours 0–6 (as low as -$15/MWh at hour 2), consistent with mild weather and wind oversupply on that specific day. The "2022 six-fold difference" framing undersells the anomaly: hour 0 in 2022 is approximately 30× higher than hour 8 in 2023 ($450 vs. $2.3). DA LMP by month: To average DA LMP within each month for Chicago Gen Hub, use data_CGH (all days, not just holidays) using the dataset created in Q13

```
# data_CGH[, .(avg_da_lmp = mean(total_lmp_da, na.rm = TRUE)), by = month][order(avg_da_lmp)]
```

DA LMP by month results: December is a clear outlier at $73.21/MWh, followed by summer months (Oct $49.55, Sep $46.00, Jul $44.19, Jun $43.27). Winter holiday months (Jan $27.09, Feb $38.99) are much lower.

Month Avg DA LMP ($/MWh) Note Jan $27.09 New Year's Day, MLK Day Feb $38.99 Presidents' Day May $30.14 Memorial Day Jun $43.27 Juneteenth Jul $44.19 Independence Day Sep $46.00 Labor Day Oct $49.55 Columbus Day Nov $30.04 Veterans Day, Thanksgiving Dec $73.21 * Christmas (winter heating + holiday demand spike)

Hypothesis: DA LMP exhibits a dual-peak seasonal structure driven by HVAC load: summer cooling demand peaks in June–August and winter heating demand peaks in December–January, with the spring/fall shoulder season (March–May, September–October) at lower levels. The December premium also reflects reduced generation availability and scarcity-pricing dynamics coincident with holiday scheduling.

How to check: Plot avg_da_lmp by month and year using data_CGH (as done in Q15). If the summer and December peaks persist across all three years — not just 2022 — the hypothesis is supported by structural demand factors rather than being an artifact of the energy crisis alone.

```r
xmas_PJM[month == 'Dec' &
               pnode_name == 'CHICAGO GEN HUB' &
               am_pm == 'AM',
  .(average_lmp_rt = mean(total_lmp_rt)), by = .(hour, year)] %>%
  .[order(year)]
```

```
##      hour  year average_lmp_rt
##     <int> <int>          <num>
##  1:    11  2021      27.704718
##  2:    10  2021      19.957507
##  3:     9  2021      18.752096
##  4:     8  2021      19.215571
##  5:     7  2021      19.406051
##  6:     6  2021      18.283769
##  7:     5  2021      18.253173
##  8:     4  2021      19.724827
##  9:     3  2021      16.290776
## 10:     2  2021      16.157507
## 11:     1  2021      15.966148
## 12:     0  2021      15.079724
## 13:     0  2022     450.443064
## 14:     1  2022     145.468962
## 15:     2  2022      82.957333
## 16:     3  2022      84.570147
## 17:     4  2022      88.248558
## 18:     5  2022      97.736789
## 19:     6  2022     102.542404
## 20:     7  2022     142.455679
## 21:     8  2022     126.036833
## 22:     9  2022     100.580731
## 23:    10  2022      72.838327
## 24:    11  2022     139.631769
## 25:     0  2023      -0.718173
## 26:     1  2023      -4.343641
## 27:     2  2023     -15.114128
## 28:     3  2023      -4.398942
## 29:     4  2023     -14.180320
## 30:     5  2023      -4.575865
## 31:     6  2023      -1.091859
## 32:     7  2023       1.596038
## 33:     8  2023       2.339558
## 34:     9  2023       5.676032
## 35:    10  2023       8.173750
## 36:    11  2023       5.831167
##      hour  year average_lmp_rt
```

IV. Introduction to Plotting

13. Create a dataset called "data_CGH," restricted to the "'CHICAGO GEN HUB." What does the number of observations reflect?

```r
data_CGH <- data_PJM[pnode_name == "CHICAGO GEN HUB"]
nrow(data_CGH)
```

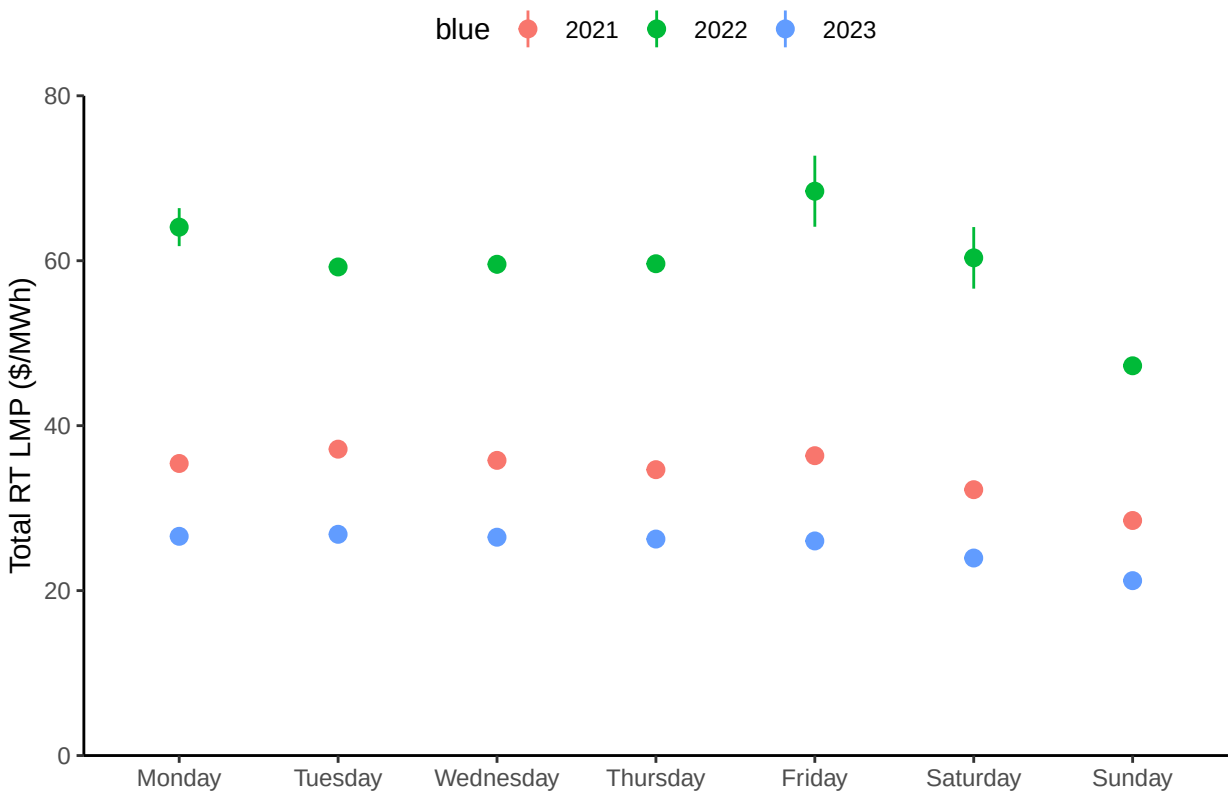
```
## [1] 26280
```

The number of observations reflects the total count of hourly pricing records for the Chicago Gen Hub node across all years in the dataset. Since PJM data is hourly and the dataset spans ~3 years (2021–2023), we expect roughly 24 hours x 365 days x 3

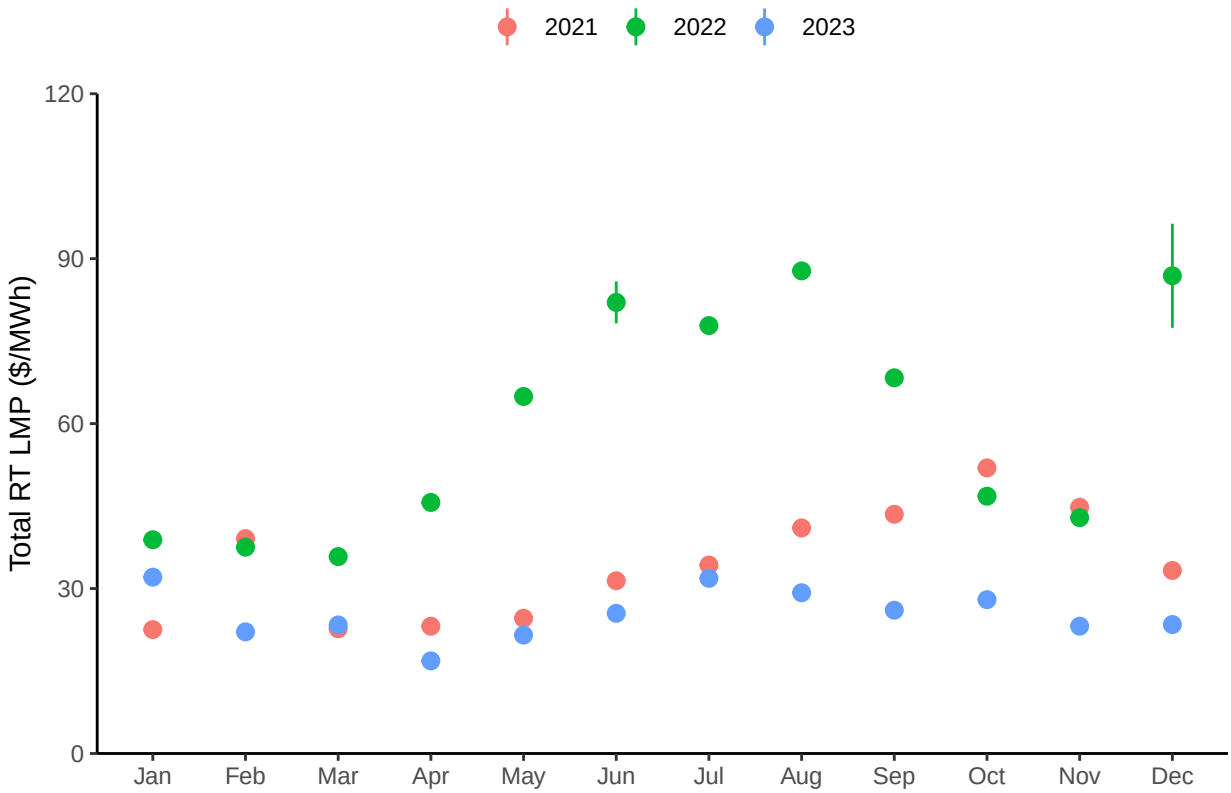
years ~ 26,280 rows per node. This hourly granularity is the fundamental unit of observation in wholesale electricity markets — the DA and RT auctions both clear on an hourly basis.

14. The code below generates Figure 1. What stands out? What is each point on the plot averaging over? Interpret the resulting figure when `color = factor(year)` is removed. Try plotting RT LMP by month and by hour instead of day-of-week.

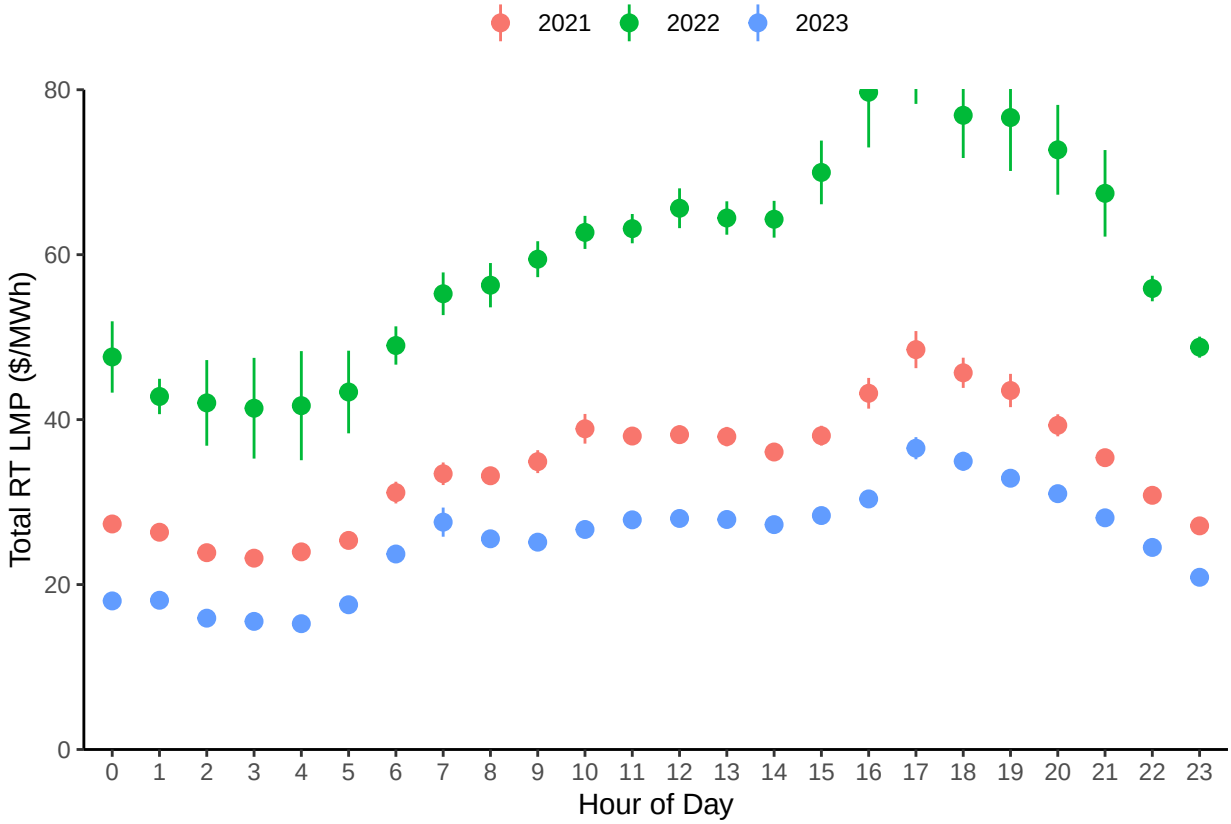
```r
data_CGH$day_of_week <- factor(data_CGH$day_of_week,
  levels = c("Monday","Tuesday","Wednesday","Thursday","Friday","Saturday","Sunday"))

# By day-of-week
ggplot(data_CGH[order(day_of_week)], aes(x = day_of_week, y = total_lmp_rt,
                                         color = factor(year))) +
  stat_summary() +
  labs(color = 'blue', x = '', y = 'Total RT LMP ($/MWh)') +
  coord_cartesian(ylim = c(0, 80)) +
  scale_y_continuous(expand = c(0, 0)) +
  theme_classic() +
  theme(legend.position = 'top')
```




```r
# By month
data_CGH$month <- factor(data_CGH$month, levels = month.abb)
ggplot(data_CGH, aes(x = month, y = total_lmp_rt, color = factor(year))) +
  stat_summary() +
  labs(color = '', x = '', y = 'Total RT LMP ($/MWh)') +
  coord_cartesian(ylim = c(0, 120)) +
  scale_y_continuous(expand = c(0, 0)) +
  theme_classic() +
  theme(legend.position = 'top')
```




```r
# By hour
ggplot(data_CGH, aes(x = factor(hour), y = total_lmp_rt, color = factor(year))) +
  stat_summary() +
  labs(color = '', x = 'Hour of Day', y = 'Total RT LMP ($/MWh)') +
  coord_cartesian(ylim = c(0, 80)) +
  scale_y_continuous(expand = c(0, 0)) +
  theme_classic() +
  theme(legend.position = 'top')
```

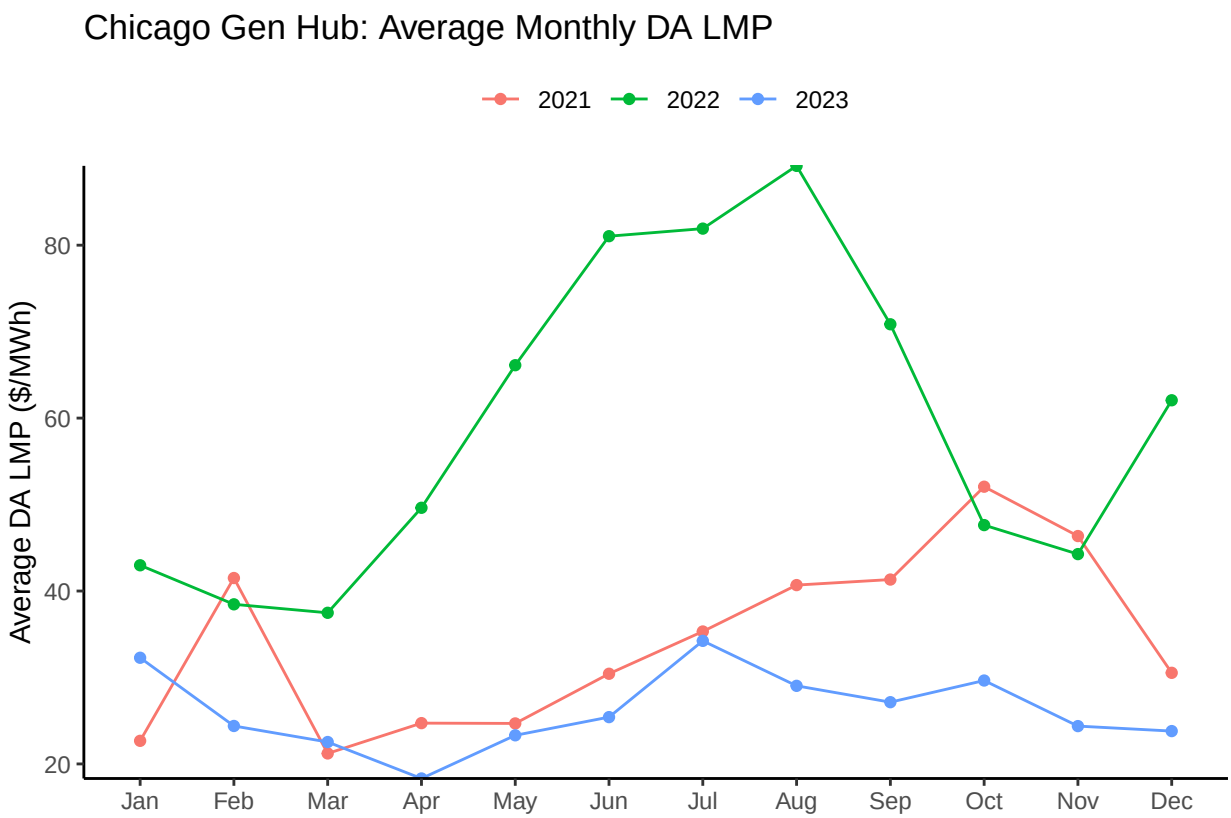


By hour, the overnight trough (hours 2–4 AM) is consistently the lowest-price period across all years, reflecting minimal industrial and commercial load. However, the peak structure differs significantly by year. In 2023, the profile most closely resembles the classic dual-ramp pattern: prices step up around hour 6 as commercial load comes online and again around hour 17–18 as evening demand builds. In 2021, the profile is relatively flat after the overnight trough, with no pronounced evening spike at the annual-average level — individual high-price events average out. In 2022, the most striking feature is a near-monotonic increase from the overnight trough through hour 19–21, where the annual average peaks above $65/MWh. This shape reflects the dominance of summer afternoon gas peaker pricing in 2022: the gas price shock was most severe during peak afternoon demand hours, lifting the conditional mean so sharply that it overwhelms the typical morning-ramp/evening-peak bimodality. The result is that the "morning ramp" and "evening peak" pattern described by the duck curve is only clearly visible in years with more normal supply-cost conditions.

Removing `color = factor(year)` collapses across years into a single average, obscuring the fact that 2022 was a structurally different price regime. By month, summer (Jun–Aug) and winter peaks (Jan–Feb) emerge — classic dual-peak demand seasonality driven by cooling and heating loads. By hour, prices are highest in the morning ramp (7–9am) and evening peak (5–8pm), lowest in the overnight trough (2–4am), reflecting the diurnal demand cycle described by the duck curve.

15. Write code that plots average monthly day-ahead LMP for "CHICAGO GEN HUB." What happened during the summer of 2022?

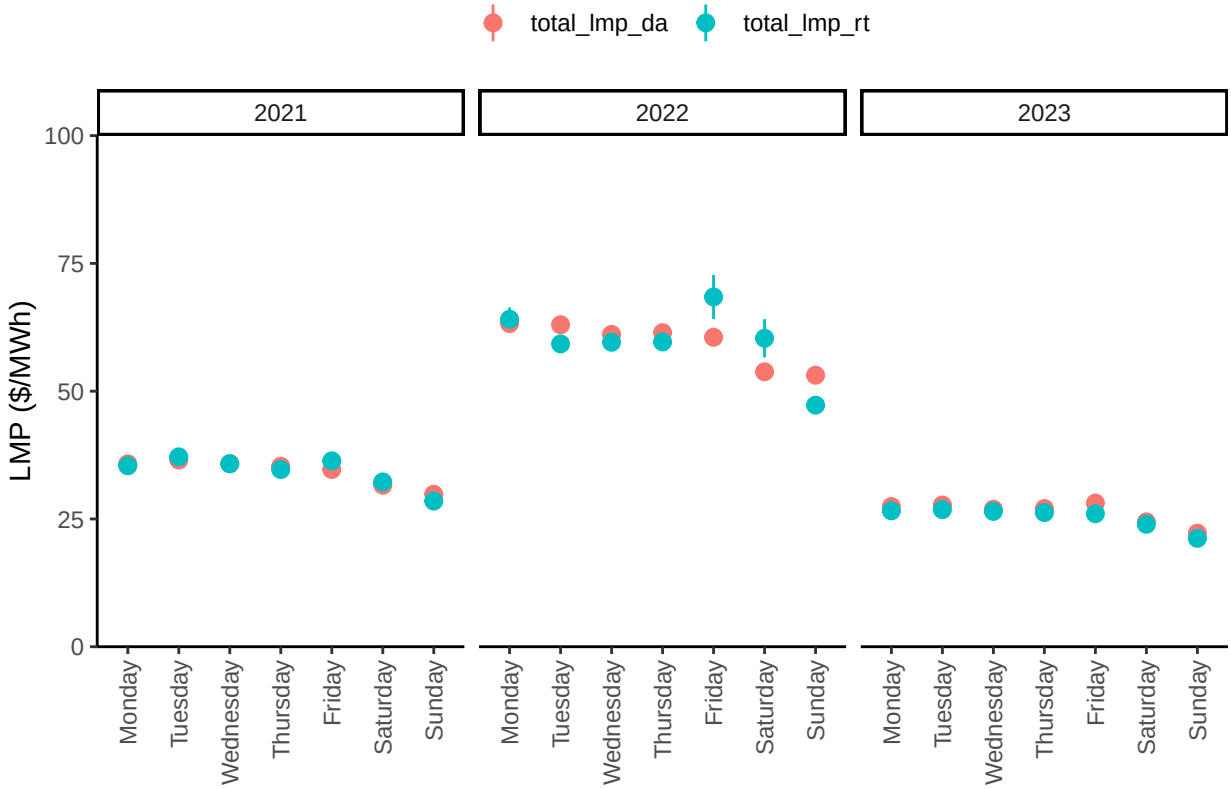
```
CGH_monthly_da <- data_CGH[, .(avg_da_lmp = mean(total_lmp_da, na.rm = TRUE)),
                               by = .(year, month)]
CGH_monthly_da$month <- factor(CGH_monthly_da$month, levels = month.abb)

ggplot(CGH_monthly_da, aes(x = month, y = avg_da_lmp,
                           color = factor(year), group = factor(year))) +
  geom_line() +
  geom_point() +
  labs(color = '', x = '', y = 'Average DA LMP ($/MWh)',
       title = 'Chicago Gen Hub: Average Monthly DA LMP') +
  scale_y_continuous(expand = c(0, 0)) +
  theme_classic() +
  theme(legend.position = 'top')
```



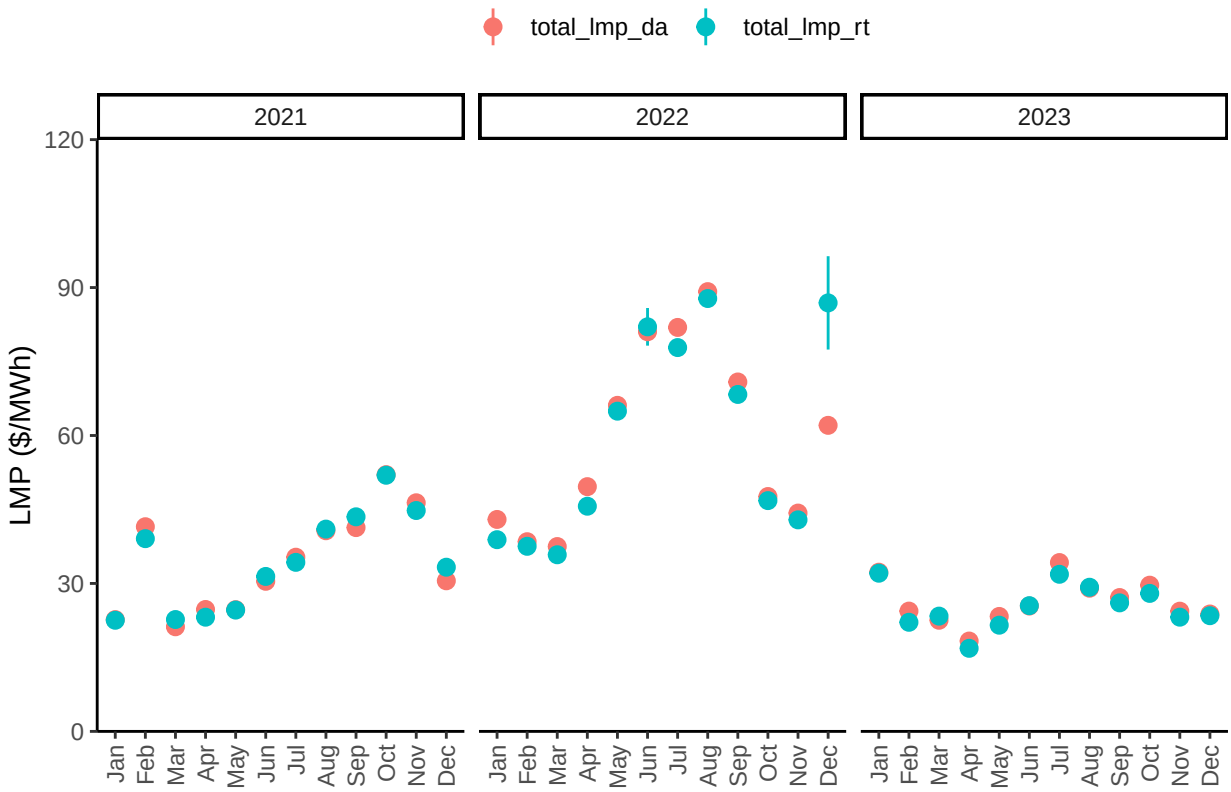
Summer 2022 saw a dramatic spike in DA LMP — roughly 2–3x higher than corresponding months in 2021 or 2023. This reflects the global energy crisis triggered by Russia's invasion of Ukraine, which sharply elevated natural gas prices. Since natural gas turbines (combined-cycle and peakers) are typically the marginal unit setting the LMP during peak demand hours, the spark spread widened substantially: higher gas input costs translated directly into higher electricity clearing prices. Concurrent extreme heat in the Midwest drove cooling demand to elevated levels, placing additional upward pressure on LMPs.

16. Oftentimes, pivoting data from "wide" to "long" is helpful for plotting purposes. The following, for example, pivots the "data_CGH" by: [1] creating a "lmp_type" column that stores the names "total_lmp_rt," "total_lmp_da," and "prev_lmp_da" as entries; and [2] creating a corresponding "values" column that stores values of those series. The second block of code plots and compares the "total_lmp_rt" series and "total_lmp_da" series by day-of-the-week, as well as by year. What happens when you replace "day_of_week" with month?

```
data_CGH_long <- data_CGH[, .(year, month, hour, day_of_week,
                              total_lmp_rt, total_lmp_da)] %>%
  pivot_longer(cols = !c(year, month, hour, day_of_week),
               names_to  = 'lmp_type',
               values_to = 'values')
setDT(data_CGH_long)

# By day-of-week
ggplot(data_CGH_long, aes(x = day_of_week, y = values,
                          color = factor(lmp_type))) +
  stat_summary() +
  labs(color = '', x = '', y = 'LMP ($/MWh)') +
  coord_cartesian(ylim = c(0, 100)) +
  scale_y_continuous(expand = c(0, 0)) +
  theme_classic() +
  theme(legend.position = 'top',
        axis.text.x = element_text(angle = 90, vjust = 0.5, hjust = 1)) +
  facet_wrap(~year)
```



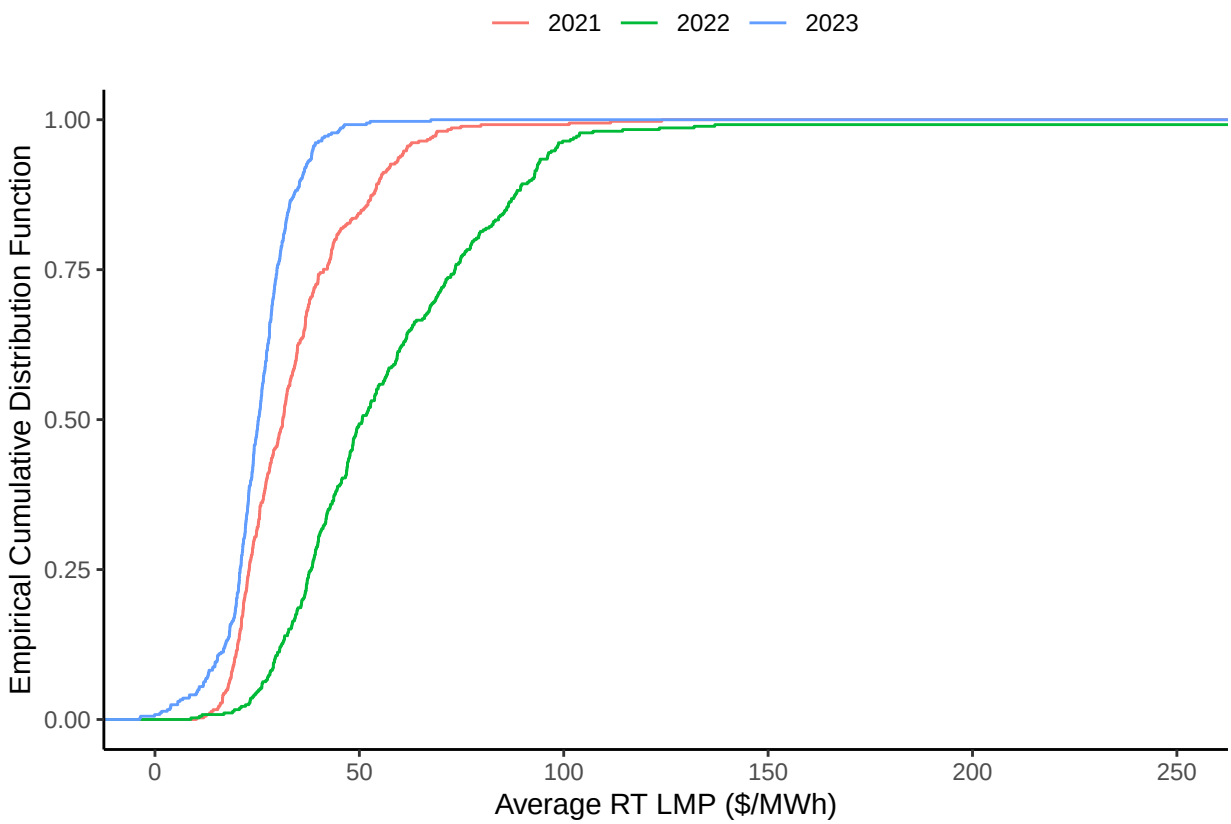

```r
# By month
data_CGH_long$month <- factor(data_CGH_long$month, levels = month.abb)
ggplot(data_CGH_long, aes(x = month, y = values,
                          color = factor(lmp_type))) +
  stat_summary() +
  labs(color = '', x = '', y = 'LMP ($/MWh)') +
  coord_cartesian(ylim = c(0, 120)) +
  scale_y_continuous(expand = c(0, 0)) +
  theme_classic() +
  theme(legend.position = 'top',
        axis.text.x = element_text(angle = 90, vjust = 0.5, hjust = 1)) +
  facet_wrap(~year)
```



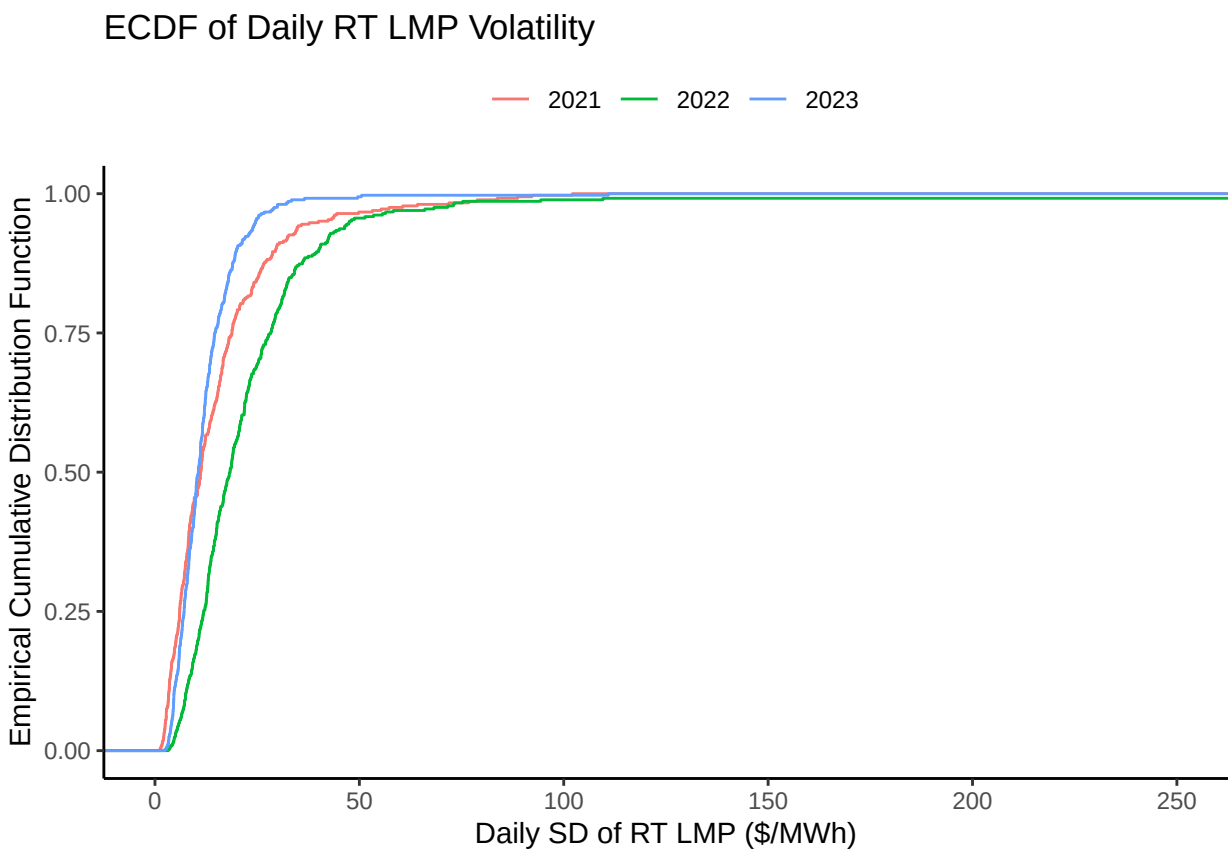
Replacing `day_of_week` with `month` reveals that RT and DA LMPs track each other closely on average, but the RT-DA spread (basis) is widest during summer months (Jun–Aug) and winter stress periods (Dec–Feb). This is precisely when demand is most uncertain and weather-driven, making the DA forecast least reliable. The DA market clears using a deterministic unit commitment that cannot perfectly anticipate real-time conditions — particularly intermittent renewables or unexpected demand spikes. Summer 2022 shows the largest divergence (with the exception of December 2022), consistent with the gas price shock amplifying RT volatility relative to the forward-looking DA signal. Notably in December 2022, the bomb cyclone event created an extreme RT-DA divergence that is visible as a separated teal/red pair in the Dec panel — this is the same event identified in Q21 as the most extreme spread-volatility day.

17. The following code calculates average RT and DA LMPs for each day in the CGH dataset and then plots the empirical cumulative distribution (ECDF) of average RT LMPs by year. Note that the series corresponding to 2022 is below that for 2021 and 2023, indicating that for each possible value x, there were more days in 2021 and 2023 for which the average daily RT LMP was less than x; in other words, RT LMP was higher in 2022 than in 2021 or 2023. This relationship between probability distributions is known as first-order stochastic dominance (FOSD). Adjust the code below such that it also calculates the daily volatility of RT LMP, using the `sd()` function, and identify another example of FOSD.

```
CGH_summary_daily <- data_CGH[, .(mean_rt_lmp = mean(total_lmp_rt),
                                  mean_da_lmp  = mean(total_lmp_da),
                                  sd_rt_lmp    = sd(total_lmp_rt)),
                               by = .(day, month, year)]

# ECDF of mean daily RT LMP by year
ggplot(CGH_summary_daily, aes(mean_rt_lmp, color = factor(year))) +
  stat_ecdf() +
  coord_cartesian(xlim = c(0, 250)) +
  labs(color = '', x = 'Average RT LMP ($/MWh)',
       y = 'Empirical Cumulative Distribution Function') +
  theme_classic() +
  theme(legend.position = 'top')
```




```r
# ECDF of daily RT LMP volatility by year
ggplot(CGH_summary_daily, aes(sd_rt_lmp, color = factor(year))) +
  stat_ecdf() +
  coord_cartesian(xlim = c(0, 250)) +
  labs(color = '', x = 'Daily SD of RT LMP ($/MWh)',
       y = 'Empirical Cumulative Distribution Function',
       title = 'ECDF of Daily RT LMP Volatility') +
  theme_classic() +
  theme(legend.position = 'top')
```

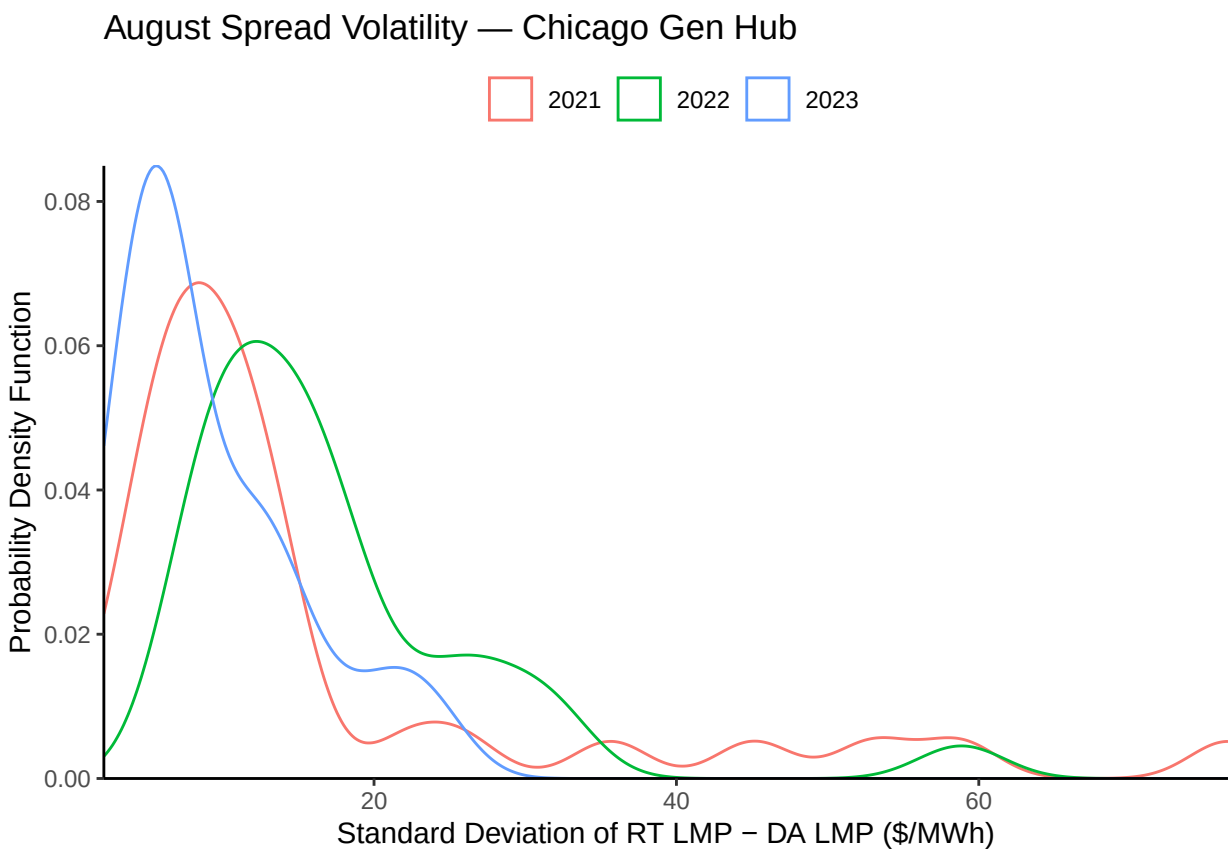


Adding `sd_rt_lmp` captures intra-day RT price volatility, which is conceptually the spread risk that a storage or CfD holder faces within each day. The second ECDF plot provides another example of FOSD: 2021 and 2023 jointly first-order stochastically dominate 2022 in terms of daily RT LMP volatility — the 2022 ECDF lies uniformly to the right, confirming that 2022 had higher intra-day spread risk at every quantile. For every threshold level of intra-day price volatility, a larger fraction of 2021 and 2023 days fell below that level. This confirms that 2022 was not only a high-price year but a high-volatility year.

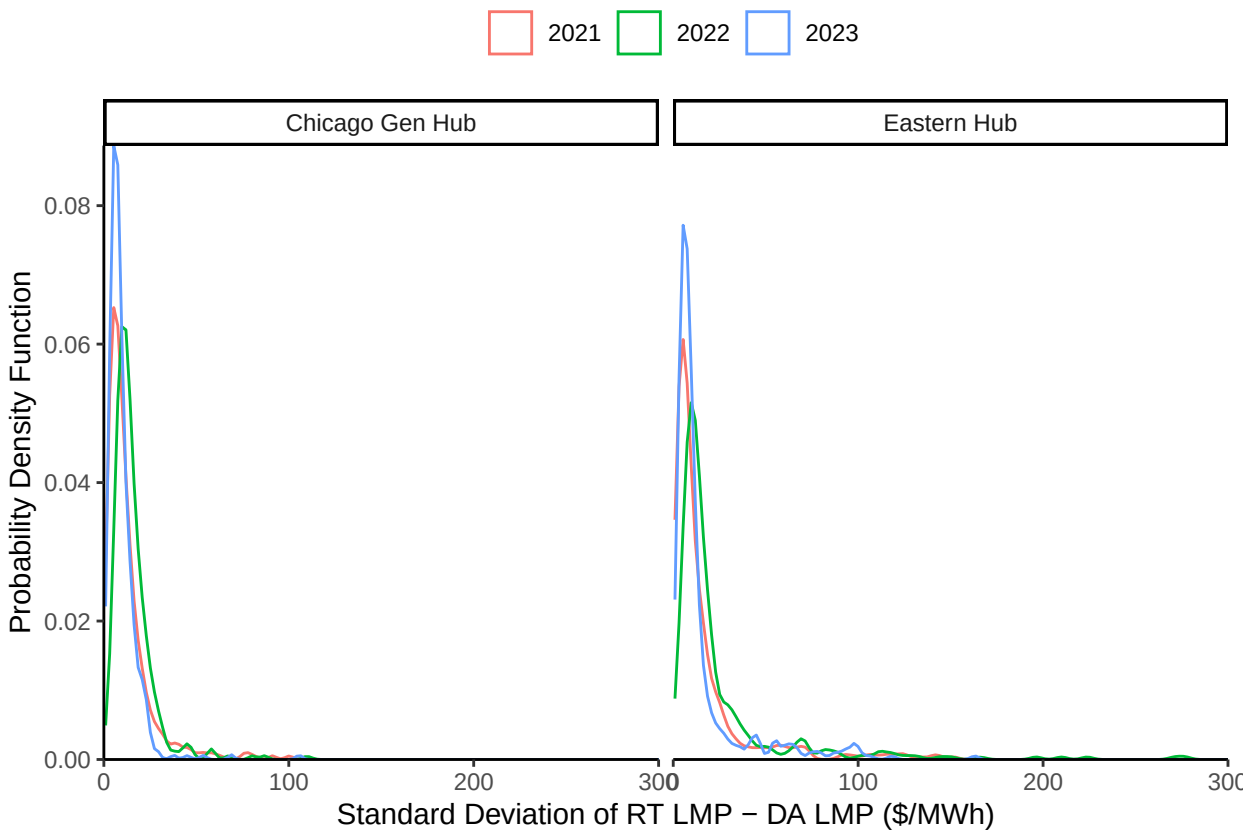
V. Pricing Anomalies and Patterns

18. The "spread" field in the PJM data reflects "total_lmp_rt" less "total_lmp_da." The code below calculates the volatility of this spread for each observation of the data, for CHICAGO GEN HUB, and plots the density of this volatility for August of each year. In which year was the volatility of the LMP spread more variable? Use the PJM data to show how the distribution of volatility of the RT-DA LMP spread for the Chicago Gen Hub compares to that for the Eastern Hub ("EASTERN HUB").

```
vol_daily_CGH <- data_CGH[, .(vol_spread = sd(spread, na.rm = TRUE)),
                              by = .(day, month, year)]

# August density by year - Chicago Gen Hub
ggplot(vol_daily_CGH[month == 'Aug'], aes(x = vol_spread, color = factor(year))) +
  geom_density() +
  labs(x = 'Standard Deviation of RT LMP - DA LMP ($/MWh)',
       y = 'Probability Density Function', color = '',
       title = 'August Spread Volatility - Chicago Gen Hub') +
  scale_x_continuous(expand = c(0, 0)) +
  scale_y_continuous(expand = c(0, 0)) +
  theme_classic() +
  theme(legend.position = 'top')
```



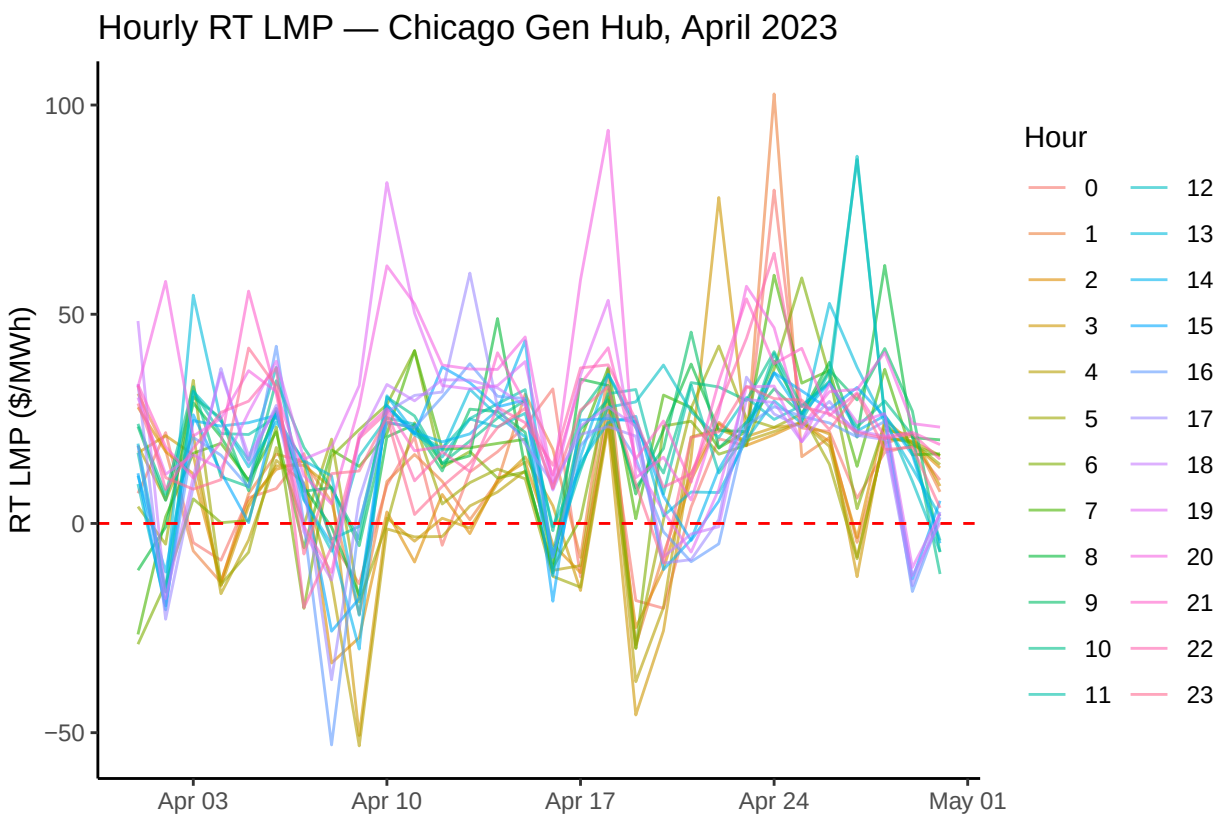

```r
# Eastern Hub comparison
data_EH <- data_PJM[pnode_name == 'EASTERN HUB']
vol_daily_EH <- data_EH[, .(vol_spread = sd(spread, na.rm = TRUE)),
                          by = .(day, month, year)]

vol_compare <- rbind(
  vol_daily_CGH[, node := 'Chicago Gen Hub'],
  vol_daily_EH[,  node := 'Eastern Hub']
)

ggplot(vol_compare, aes(x = vol_spread, color = factor(year))) +
  geom_density() +
  coord_cartesian(xlim = c(0, 300)) +
  labs(x = 'Standard Deviation of RT LMP - DA LMP ($/MWh)',
       y = 'Probability Density Function', color = '') +
  scale_x_continuous(expand = c(0, 0)) +
  scale_y_continuous(expand = c(0, 0)) +
  theme_classic() +
  theme(legend.position = 'top') +
  facet_wrap(~node)
```



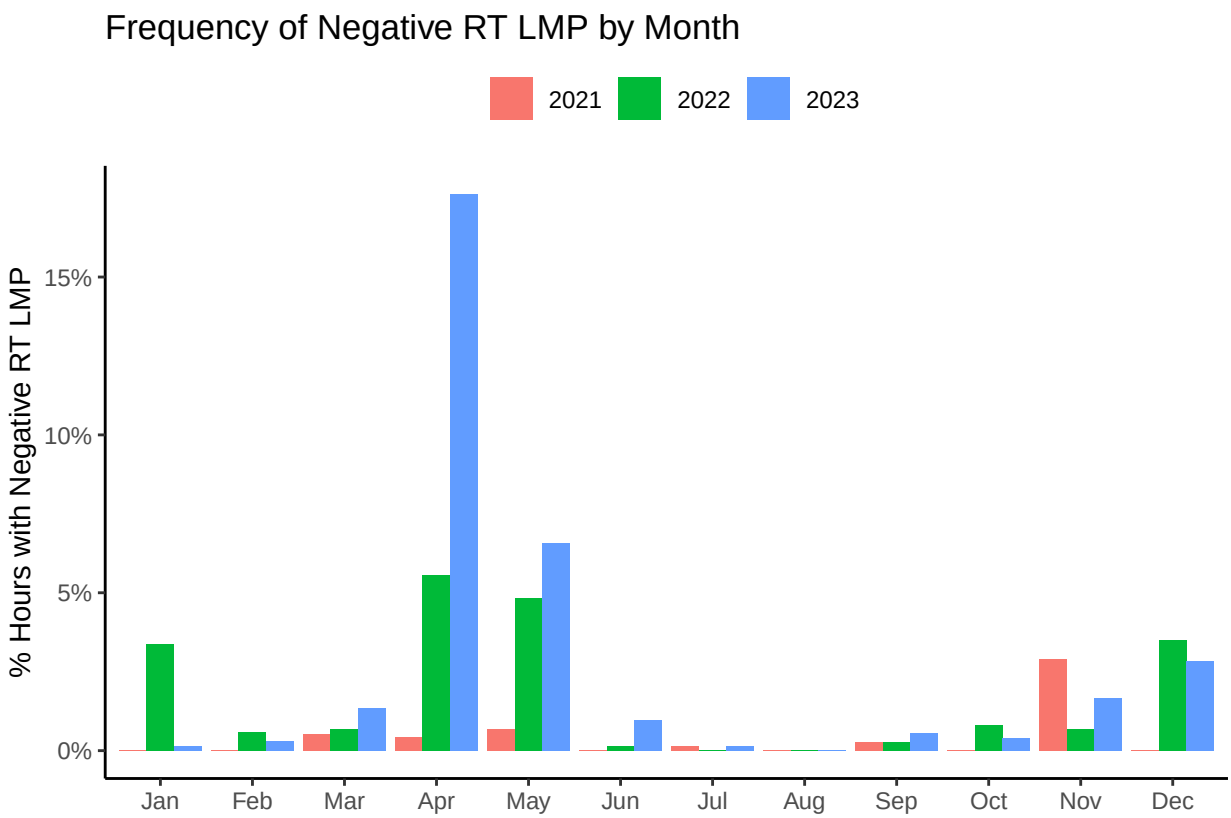
The node comparison reveals a qualitative difference in market structure. At Chicago Gen Hub, spread volatility distributions are clearly year-differentiated: 2022 (green) is the flattest and most right-skewed, with substantial probability mass extending past \$40/MWh and a visible secondary hump near \$50–60/MWh — the signature of the gas price shock amplifying intraday RT-DA divergences. In 2023 (blue) and 2021 (red), distributions are tighter but still visibly distinct, with 2023 being the most concentrated (peaking near \$7/MWh). At Eastern Hub, by contrast, the three year-curves are nearly superimposed: all peak sharply near \$2–4/MWh and have negligible mass above \$15/MWh. The year-to-year variation that dominated Chicago Gen Hub is essentially absent. This reflects two structural differences: first, Eastern Hub is an aggregated hub node on the PJM eastern seaboard with access to a highly diverse generation portfolio (gas, nuclear, coal, imports from NYISO/MISO), reducing congestion-driven RT spikes; second, because it is not located at a Midwest wind-interconnect chokepoint, the wind-intermittency contribution to spread volatility that amplifies Chicago's basis risk is absent. The practical implication is that RT-DA spread risk — and therefore the CfD basis risk faced by generators and load-serving entities — is structurally higher and more year-sensitive at Chicago Gen Hub than at Eastern Hub, making weather derivatives and transmission rights more valuable for Chicago-exposed positions.

19. Plot hourly RT LMP by day for the month of April 2023 (for Chicago Gen Hub). Notice that RT LMP was often negative. What would negative prices mean? Try identifying other months or occasions for which RT LMP was negative and form a conjecture for why these negative prices occurred.

```
# Hourly RT LMP for April 2023
ggplot(data_CGH[month == 'Apr' & year == 2023],
       aes(x = date, y = total_lmp_rt, color = factor(hour))) +
  geom_line(alpha = 0.6) +
  geom_hline(yintercept = 0, linetype = 'dashed', color = 'red') +
  labs(x = '', y = 'RT LMP ($/MWh)', color = 'Hour',
       title = 'Hourly RT LMP - Chicago Gen Hub, April 2023') +
  theme_classic() +
  theme(legend.position = 'right')
```



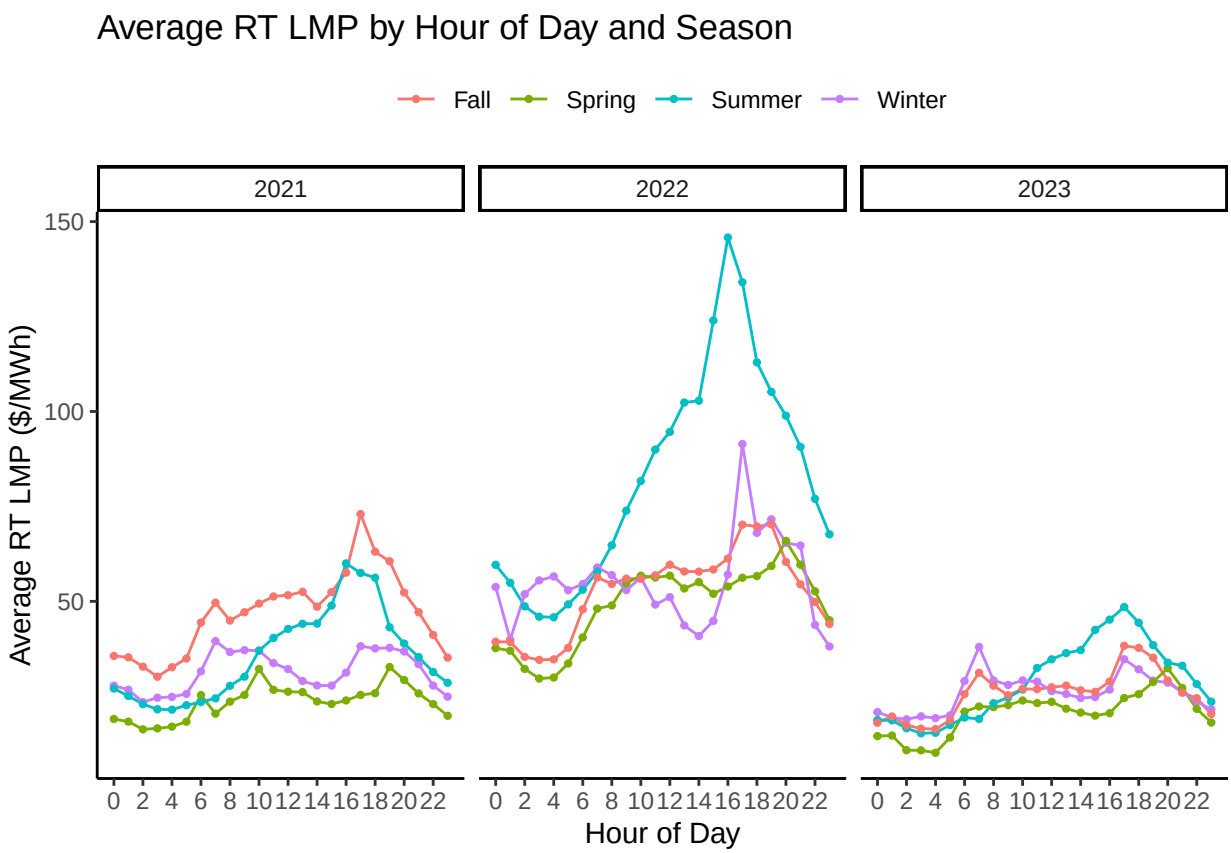

```r
# Frequency of negative RT LMP by month and year
neg_prices <- data_CGH[, .(pct_negative = mean(total_lmp_rt < 0, na.rm = TRUE),
                           n_negative   = sum(total_lmp_rt < 0, na.rm = TRUE)),
                       by = .(month, year)]
neg_prices$month <- factor(neg_prices$month, levels = month.abb)

ggplot(neg_prices, aes(x = month, y = pct_negative, fill = factor(year))) +
  geom_bar(stat = 'identity', position = 'dodge') +
  scale_y_continuous(labels = scales::percent_format()) +
  labs(x = '', y = '% Hours with Negative RT LMP', fill = '',
       title = 'Frequency of Negative RT LMP by Month') +
  theme_classic() +
  theme(legend.position = 'top')
```



Negative prices mean generators are effectively paying the grid to accept their electricity. This happens when supply exceeds demand and dispatchable units are cheaper to keep running than to shut down — operators will accept negative prices up to their shutdown cost threshold. The key drivers: (1) Wind/solar oversupply during mild-demand periods (2) Must-run baseload (3) Transmission congestion

20. Which hour of the day do you think RT LMP is highest and lowest? How would the answer depend on the season? Test your conjectures.

```
# Average RT LMP by hour and season
data_CGH[, season := fcase(
  month %in% c('Dec','Jan','Feb'), 'Winter',
  month %in% c('Mar','Apr','May'), 'Spring',
  month %in% c('Jun','Jul','Aug'), 'Summer',
  month %in% c('Sep','Oct','Nov'), 'Fall'
)]

hourly_lmp <- data_CGH[, .(mean_rt_lmp = mean(total_lmp_rt, na.rm = TRUE)),
                          by = .(hour, season, year)]

ggplot(hourly_lmp, aes(x = hour, y = mean_rt_lmp, color = season)) +
  geom_line() +
  geom_point(size = 0.8) +
  labs(x = 'Hour of Day', y = 'Average RT LMP ($/MWh)', color = '',
       title = 'Average RT LMP by Hour of Day and Season') +
  scale_x_continuous(breaks = seq(0, 23, 2)) +
  theme_classic() +
  theme(legend.position = 'top') +
  facet_wrap(~year)
```

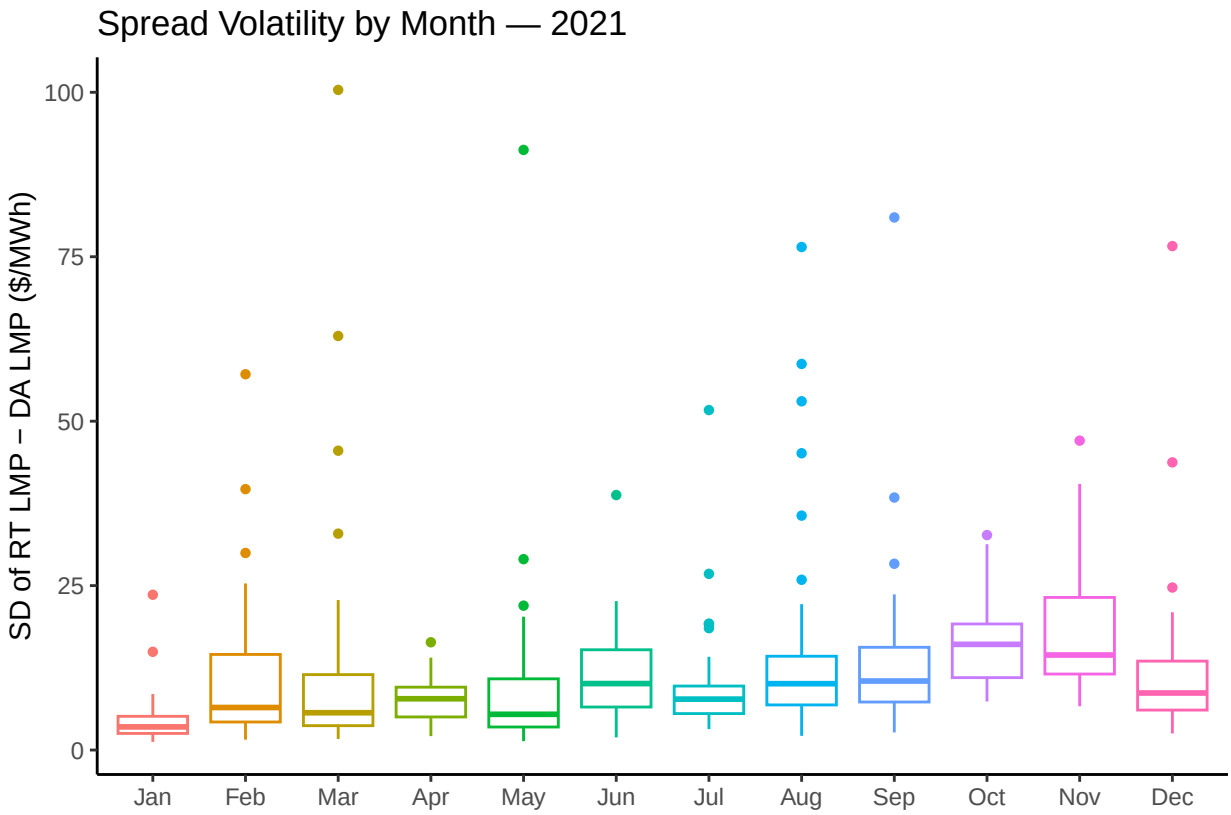


Across all seasons, RT LMP is typically lowest in the overnight trough (hours 2–5 AM) when industrial and commercial loads are minimal, and highest during the morning ramp (7–9 AM) and evening peak (5–8 PM) when demand surges. The season modulates both the magnitude and timing: in summer, the evening peak is most extreme; in winter, the morning ramp is more pronounced; in spring/fall, intra-day variation is smaller and negative-price hours are more frequent overnight.

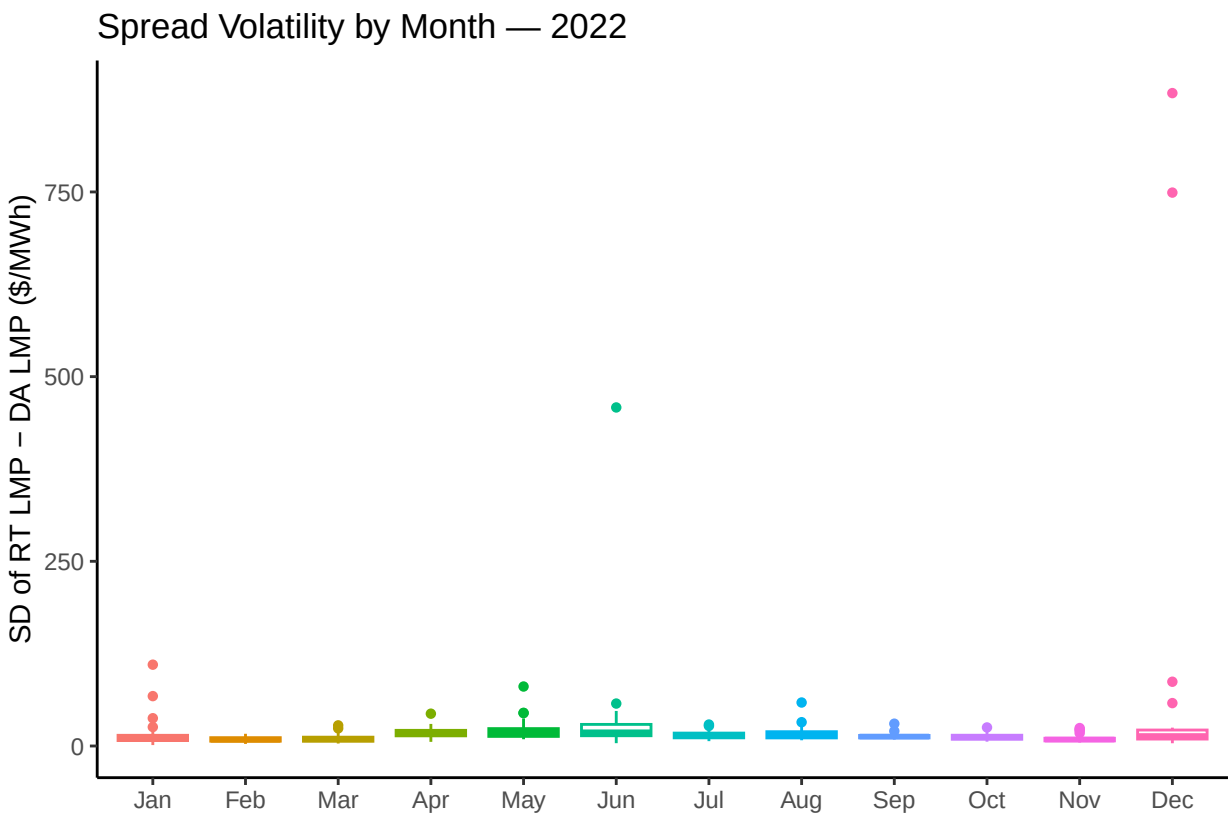
In 2022, the winter profile is an extreme outlier — average winter RT LMP approaches or exceeds $150/MWh at peak hours, driven by the December 2022 bomb cyclone (also identified in Q21). This illustrates that the hour-of-day pattern is inseparable from tail-risk events: winter mornings carry the most severe scarcity-pricing risk.

21. The code below plots boxplots of daily RT-DA LMP spread volatility for each month of 2021 for Chicago Gen Hub, such that individual points are outliers relative to the month, using the `vol_daily_CGH` dataset constructed in question 18. Plot the same figure for 2022 and 2023. What day in 2021-2023 had the most extreme RT-DA LMP spread volatility?

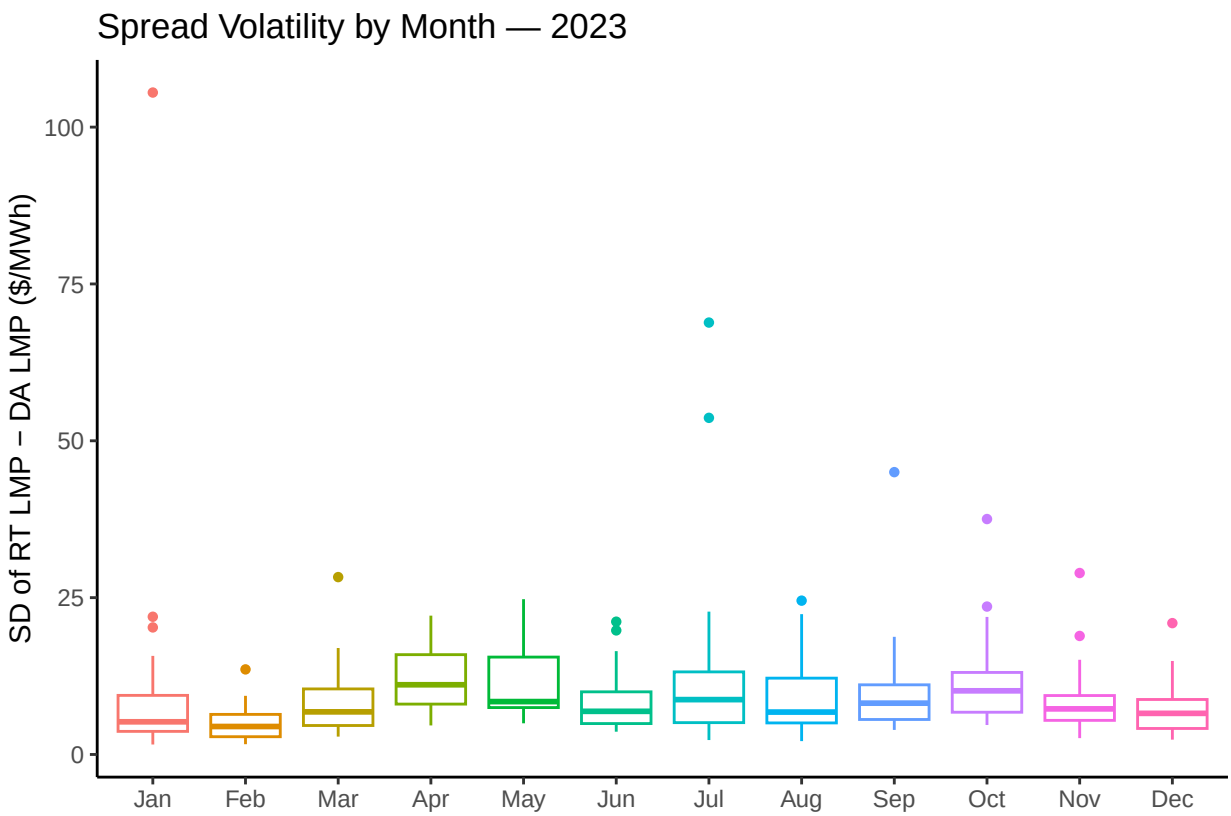
```r
# Helper function to avoid repeating code
plot_vol_boxplot <- function(yr) {
  ggplot(vol_daily_CGH[year == yr],
         aes(factor(month, levels = month.abb), vol_spread,
             color = factor(month, levels = month.abb))) +
    geom_boxplot(outlier.shape = 16, outlier.size = 1.5) +
    labs(x = '', y = 'SD of RT LMP - DA LMP ($/MWh)',
         title = paste('Spread Volatility by Month -', yr)) +
    theme(axis.text.x = element_text(angle = 90, vjust = 0.5, hjust = 1)) +
    guides(color = 'none') +
    theme_classic()
}

plot_vol_boxplot(2021)
```




```
plot_vol_boxplot(2022)
```



```r
plot_vol_boxplot(2023)
```




```r
# Find the single most extreme day across all years
vol_daily_CGH[, date_full := as.Date(paste(year, match(month, month.abb), day,
                                           sep = '-'))]
most_extreme <- vol_daily_CGH[which.max(vol_spread)]
print(most_extreme)
```

```
##      day  month  year vol_spread           node  date_full
##    <int> <fctr> <int>      <num>          <char>     <Date>
## 1:    23    Dec  2022   883.7946 Chicago Gen Hub 2022-12-23
```

The three boxplots tell a clear story of regime difference. In 2021, spread volatility was modest throughout the year — the most extreme single-day reading was approximately $100/MWh in April (likely a spring wind event), with most months having IQRs below $15/MWh. In 2023, the distribution is similarly well-behaved, with a summer peak of around $65/MWh in July. 2022 is categorically different. The December 2022 panel contains two extreme outliers near $750 and $800+/MWh that push the y-axis to a scale 8× larger than either neighboring year. Every other month's box collapses to the baseline relative to these values, illustrating that the December 2022 event was not merely an elevated year — it was a single catastrophic tail event. The precise maximum is confirmed computationally:

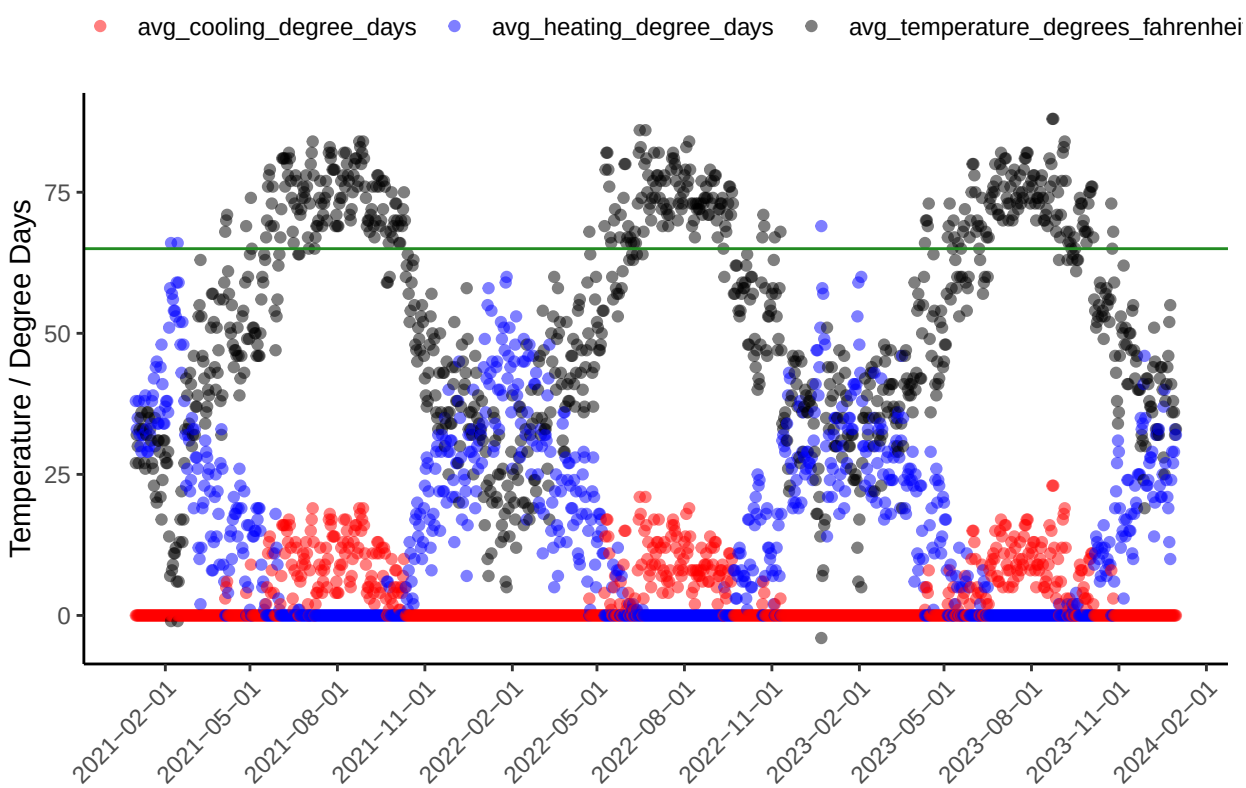
```r
most_extreme <- vol_daily_CGH[which.max(vol_spread)]
# day=23, month=Dec, year=2022, vol_spread=883.79
```

December 23, 2022 — vol_spread = $883.79/MWh — is the most extreme day in the three-year dataset by a factor of roughly 9× the next most extreme non-December-2022 observation. This was the peak of the "Elliott" bomb cyclone, a historic winter storm that simultaneously collapsed wind generation, spiked natural gas demand for heating, and caused widespread generator outages across PJM — conditions under which the RT market cleared at scarcity prices while DA commitments (made before the storm's severity was apparent) were priced at a fraction of realized real-time costs.

VI. How Weather Interacts with Energy Pricing

22. Let's turn to the dataset named "data_Chicago," which combines hourly LMP data from PJM (restricted to "CHICAGO GEN HUB") with daily weather data recorded at the Chicago O'Hare weather station. This dataset includes a number of weather-related fields, including average temperature, cooling degree days, and heating degree days. Heating degree days and cooling degree days respectively describe the number of degrees necessary to raise and lower the average temperature on a particular day to reach a fixed temperature (here, 65 degrees Fahrenheit). Note that if heating degree days is positive, then cooling degree days is zero — and vice versa. The code below plots heating degree days, cooling degree days, and average temperature for each day in 2021-2023 for Chicago. What do you notice about the pattern of heating and cooling degree days throughout a year?

```
daily_temp_Chicago <- data_Chicago[, .(date, month, year,
                                       avg_temperature_degrees_fahrenheit,
                                       avg_cooling_degree_days,
                                       avg_heating_degree_days)] %>%
  unique() %>%
  pivot_longer(cols = starts_with('avg'),
               names_to  = 'temp_series',
               values_to = 'degrees')
setDT(daily_temp_Chicago)

ggplot(daily_temp_Chicago, aes(date, degrees, color = factor(temp_series))) +
  geom_point(alpha = 0.5) +
  scale_color_manual(values = c('red', 'blue', 'black')) +
  geom_hline(yintercept = 65, color = 'forestgreen') +
  labs(x = '', color = '', y = 'Temperature / Degree Days') +
  scale_x_date(date_breaks = '3 months') +
  theme_classic() +
  theme(legend.position = 'top',
        axis.text.x = element_text(angle = 45, vjust = 1, hjust = 1))
```

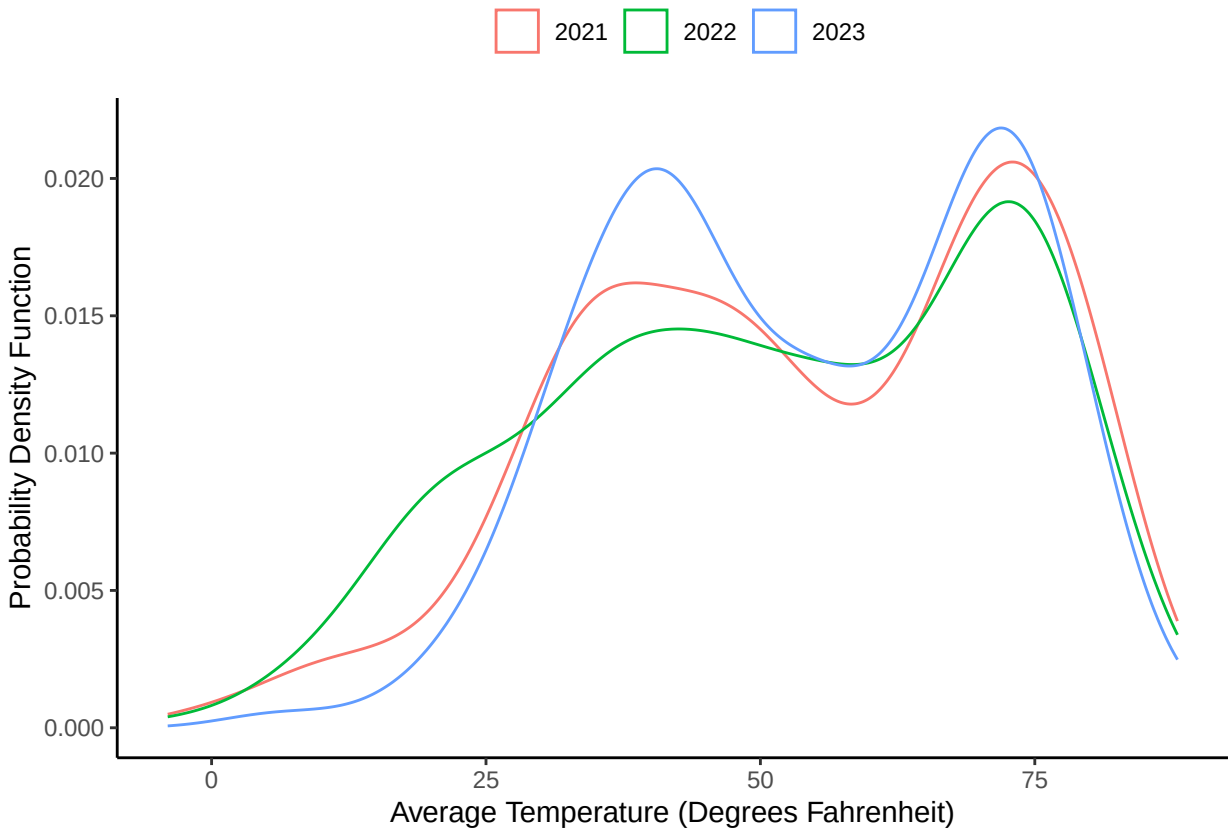


HDD and CDD are mutually exclusive by construction. When temperature exceeds 65°F, only CDD is positive; when temperature

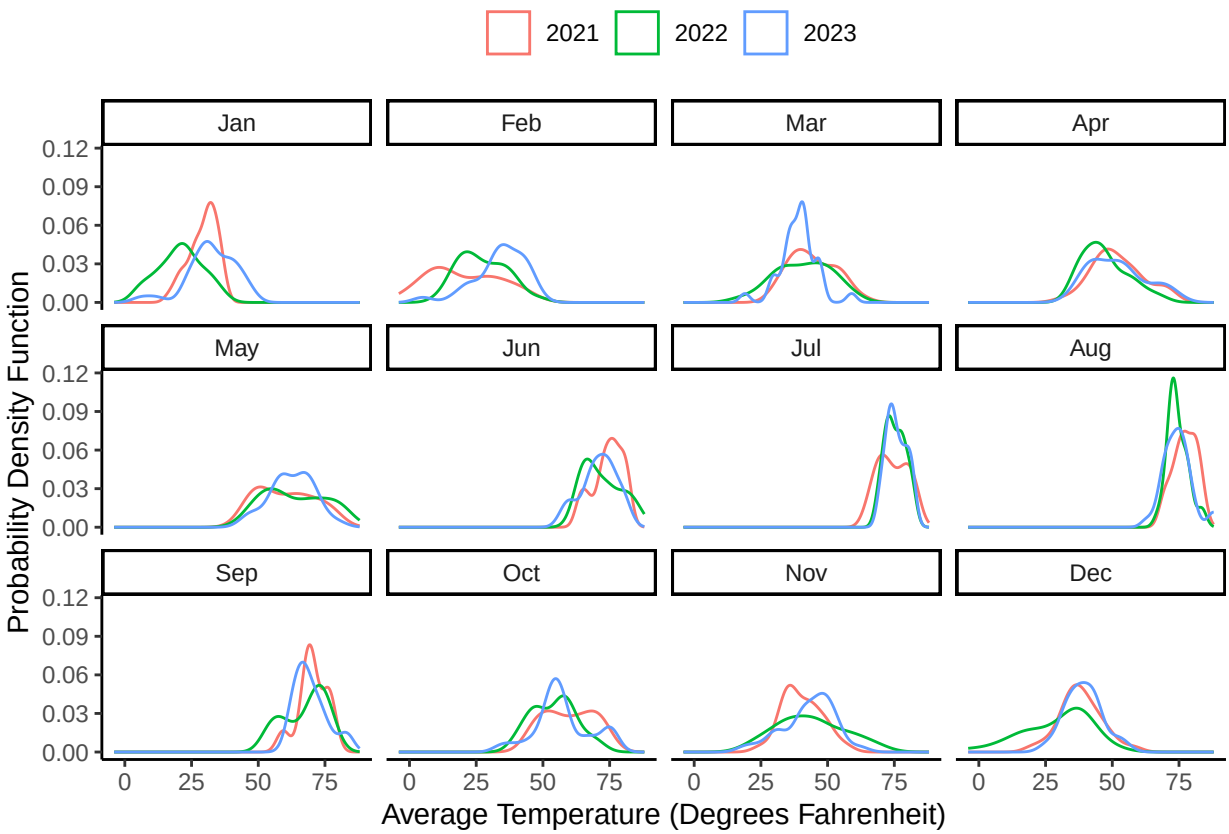
falls below 65°F, only HDD is positive. The pattern is a clear seasonal alternation: HDD dominates from November through March (Chicago winters drive large heating loads), CDD dominates from June through September (hot humid summers drive AC demand), and both are near zero in the transition months of April–May and October–November. This mirrors the dual-peak demand structure visible in the LMP data and is the econometric basis for weather derivative pricing — HDD/CDD are the standard reference indices for heating and cooling degree day contracts. The 65°F threshold (green line) is the industry-standard "base temperature" for residential HVAC thermostat behavior.

23. The code below plots the probability density function (PDF) of average daily temperature by year, using the "daily_temp_Chicago" dataset constructed above. What do you notice about the shape of temperature distributions? Form a conjecture explaining the shape and test it by comparing the PDFs by month using by adding the following command at the end of the code: `facet_wrap(~factor(month, levels = month.abb))`.

```
# Annual temperature PDFs
ggplot(daily_temp_Chicago[temp_series == 'avg_temperature_degrees_fahrenheit'],
       aes(degrees, color = factor(year))) +
  geom_density() +
  labs(color = '', x = 'Average Temperature (Degrees Fahrenheit)',
       y = 'Probability Density Function') +
  theme_classic() +
  theme(legend.position = 'top')
```



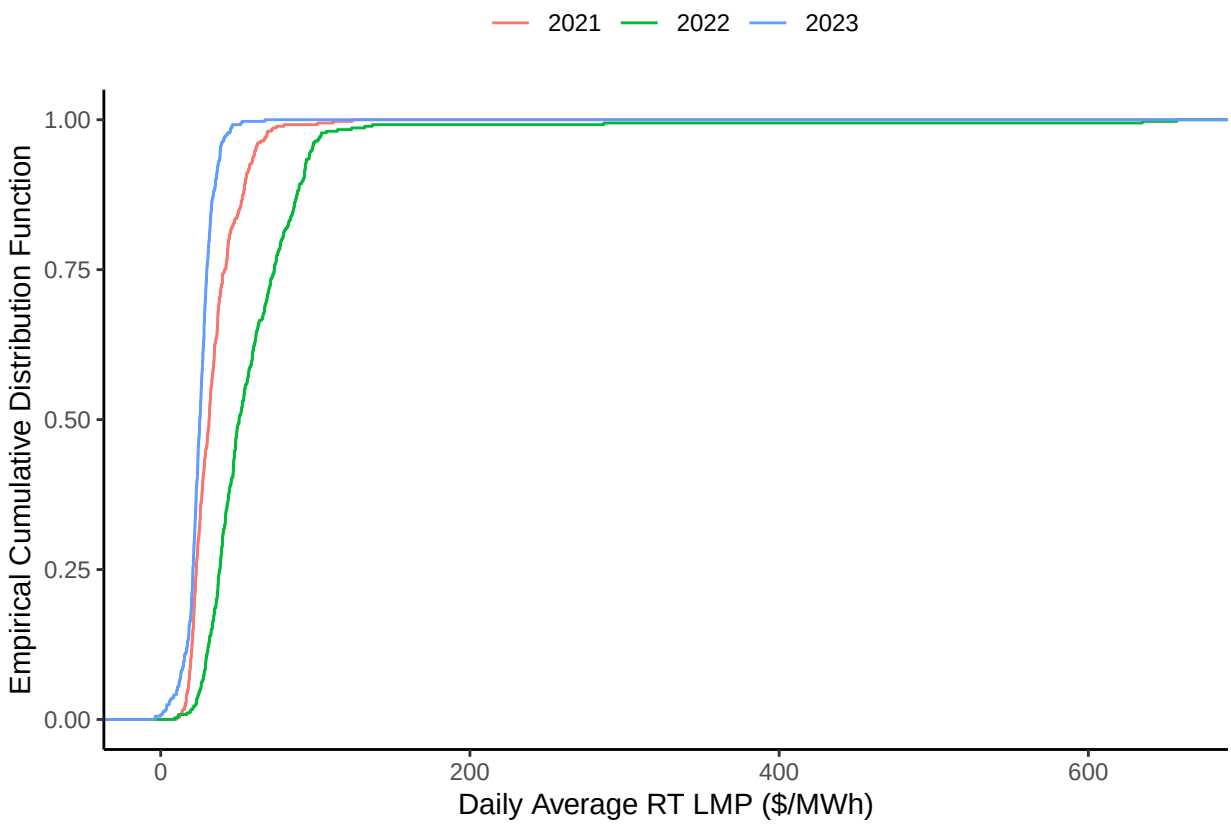

```
# By month (tests the bimodality conjecture)
ggplot(daily_temp_Chicago[temp_series == 'avg_temperature_degrees_fahrenheit'],
       aes(degrees, color = factor(year))) +
  geom_density() +
  labs(color = '', x = 'Average Temperature (Degrees Fahrenheit)',
       y = 'Probability Density Function') +
  theme_classic() +
  theme(legend.position = 'top') +
  facet_wrap(~factor(month, levels = month.abb))
```



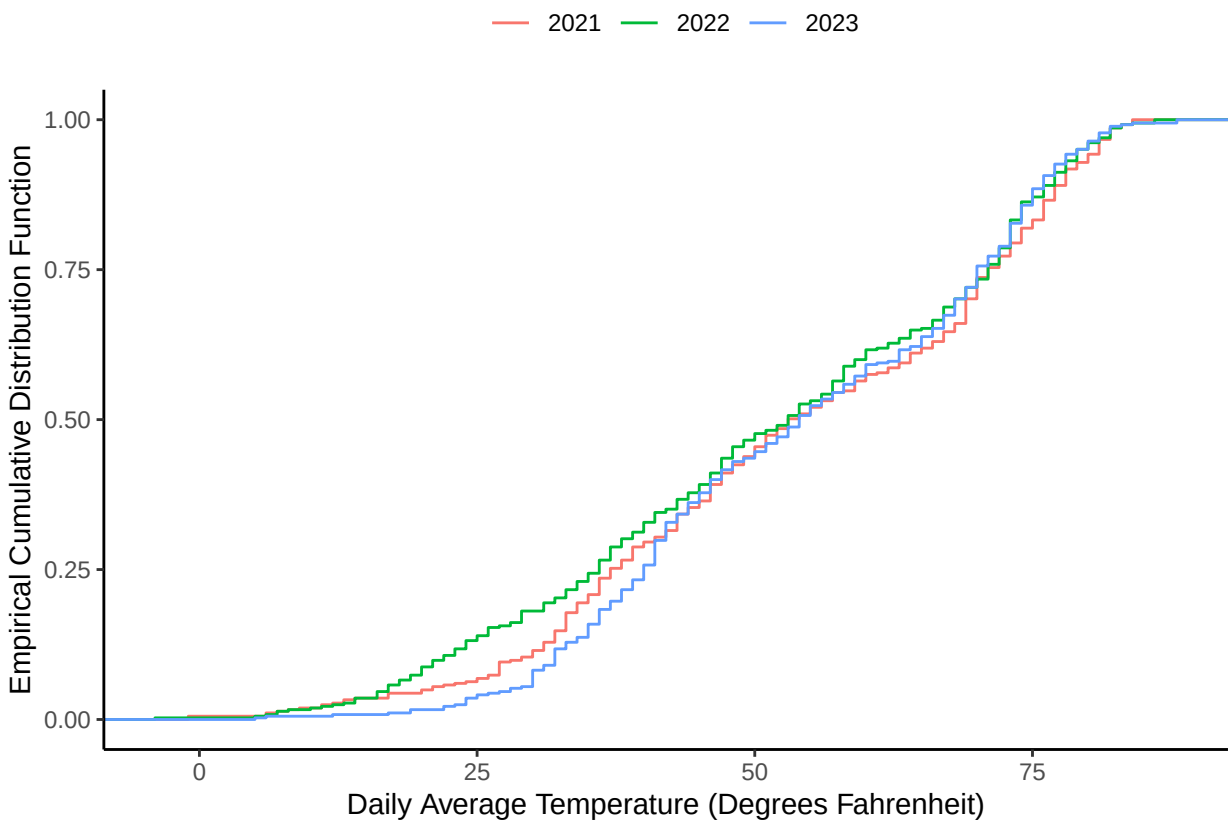
The annual temperature distribution for Chicago is bimodal — with one mode near 25–35°F (winter cluster) and another near 70–80°F (summer cluster), separated by a trough around 50–55°F. This is not consistent with normality. The conjecture is that temperatures in continental climates like Chicago spend *more time* at seasonal extremes (winter cold, summer heat) than at the spring/fall midpoints, because synoptic weather patterns are "sticky" — cold air masses and heat domes persist for weeks. The by-month facet confirms this: each individual month's temperature PDF is approximately unimodal and roughly normal, but aggregating across months creates the bimodal annual shape. This is important for LMP modeling because the **nonlinear relationship between temperature and load** (mirrored in HDD/CDD) means that the tails of the temperature distribution drive disproportionate price spikes.

24. The following code produces yearly ECDFs of average daily temperature and average daily RT LMP for Chicago. What do you notice about the relationship, in terms of FOSD, between ECDFs for the LMPs? Does the same relationship hold for temperature? Conjecture as to why temperature patterns may or may not have translated to LMP patterns. Try the same exercise, comparing by month.

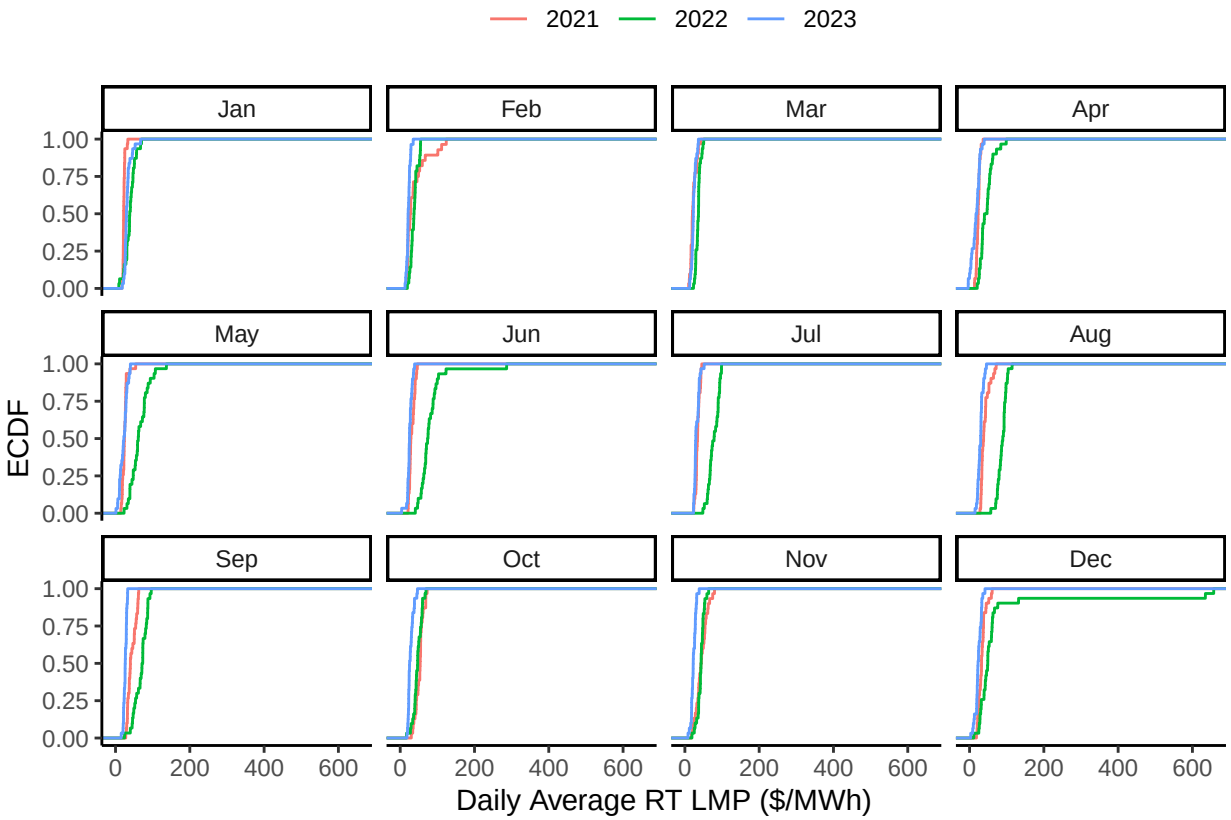
```
lmp_temp_Chicago <- data_Chicago[, .(
  mean_lmp_rt = mean(total_lmp_rt, na.rm = TRUE),
  mean_lmp_da = mean(total_lmp_da, na.rm = TRUE),
  vol_spread  = sd(spread, na.rm = TRUE),
  temp        = mean(avg_temperature_degrees_fahrenheit, na.rm = TRUE),
  humidity    = mean(relative_humidity_percent, na.rm = TRUE)
), by = .(date, month, year)]

# LMP ECDF by year
ggplot(lmp_temp_Chicago, aes(mean_lmp_rt, color = factor(year))) +
  stat_ecdf() +
  labs(x = 'Daily Average RT LMP ($/MWh)',
       y = 'Empirical Cumulative Distribution Function', color = '') +
  theme_classic() +
  theme(legend.position = 'top')
```




```r
# Temperature ECDF by year
ggplot(lmp_temp_Chicago, aes(temp, color = factor(year))) +
  stat_ecdf() +
  labs(x = 'Daily Average Temperature (Degrees Fahrenheit)',
       y = 'Empirical Cumulative Distribution Function', color = '') +
  theme_classic() +
  theme(legend.position = 'top')
```




```r
# By month - LMP
lmp_temp_Chicago$month <- factor(lmp_temp_Chicago$month, levels = month.abb)
ggplot(lmp_temp_Chicago, aes(mean_lmp_rt, color = factor(year))) +
  stat_ecdf() +
  labs(x = 'Daily Average RT LMP ($/MWh)',
       y = 'ECDF', color = '') +
  theme_classic() +
  theme(legend.position = 'top') +
  facet_wrap(~month)
```



For LMPs: the 2022 ECDF lies clearly to the right of both 2021 and 2023 — 2021 and 2023 jointly first-order stochastically dominate 2022. This confirms that 2022 RT LMPs were uniformly higher (at virtually every quantile) than the adjacent years. **For temperature**: the ECDFs for 2021, 2022, and 2023 are tightly clustered and largely overlapping — no strong FOSD relationship holds. Year-to-year temperature variation in Chicago is small relative to the seasonal range (~60–65°F swing), so annual ECDFs look nearly identical.

The disconnect between temperature FOSD and LMP FOSD is fundamental: the mapping from temperature to LMP is **highly nonlinear**. Temperature drives demand through the merit order — at moderate temperatures demand is served by baseload (cheap) units, but as temperature extremes push demand onto peaking gas turbines, marginal cost jumps sharply. The 2022 LMP premium was primarily driven by the **gas price shock** (fuel input cost rather than a temperature anomaly), illustrating that LMPs reflect both weather-driven demand *and* supply-side cost dynamics simultaneously. The by-month comparison reveals that the 2022 premium was most pronounced in the summer months, consistent with gas peakers setting the LMP during heat-driven peak demand — a direct application of the merit order / spark spread concept.

The by-month ECDFs also reveal a second 2022 premium cluster in November–December, driven by the bomb cyclone scarcity event rather than the sustained gas cost shock — illustrating that LMP FOSD across years reflects the superposition of multiple structurally distinct shocks.